OxML 2025

MLx FUNDAMENTALS

AI for Global Goals

Introduction to Machine Learning/Deep Learning

Instructor: Biwei Huang

UC San Diego

HALICIOĞLU DATA SCIENCE INSTITUTE

Course Outline

- Supervised Learning: Linear Model
- Supervised Learning: Feature Maps
- Supervised Learning: Radial Basis Function
- Neural Networks - Multilayer Perceptron
- Convolutional Neural Network
- Autoencoder
- Causality

Supervised Learning

Linear Model

Outline

- Supervised Learning
  - Setting & Technical Terms
  - General pipeline
- Empirical Risk Minimization – Linear Regression
  - Step 1: Hypothesis class
  - Step 2: Loss function
  - Step 3: Minimize empirical risk
- Extension to Linear classification

Outline

- Supervised Learning
  - Setting & Technical Terms
  - General pipeline
- Empirical Risk Minimization – Linear Regression
  - Step 1: Hypothesis class
  - Step 2: Loss function
  - Step 3: Minimize empirical risk
- Extension to Linear classification

Motivating Example

Task: Given your friends movie rating (0-10) to predict "your rating of the movie"

Training Data

Movie	friend 1 x_1	friend 2 x_2	x_3	x_4	friend 5 x_5	You y
#1	8	5	9	2	1	6
#2	3	5	7	8	2	8
#3	1	5	2	3	3	9
#4	0	5	3	8	2	?

output is a number

Task: predict what's your rating?

input/feature vector → $\vec{X} = (x_1, x_2, x_3, x_4, x_5)$ ← rating from 5 friends

$H(\vec{x}) \rightarrow$ prediction ; $H : \mathbb{R}^5 \rightarrow \mathbb{R}$

H is called Prediction/Hypothesis function

Prediction

- **Prediction** is a core ML task.

1. **Regression**: output is a "number."
   - Example: movie rating, future salary

   e.g. 6 e.g. 5k

2. **Classification**: output is a **class label**.
   - Example: like the movie? mango is ripe? (yes/no) → **binary** (0, 1) (1, 0)
   - Example: species (cat, dog, mongoose) → **multiclass**

   Class 0 1 2

Regression: $H(\vec{x}) \to \mathbb{R}$

binary classification: $H(\vec{x}) \to \{0, 1\}$

multiclass classification: $H(\vec{x}) \to \{0, 1, 2\}$

How to select Prediction Function?

- **Idea:** Use **data** to pick a prediction function that worked well in the past. (using data / history)
  - We *hope* it **generalizes** to future predictions.
    (assumption: data distribution is the same from the past & now)
  - A function that did well in the past but does not generalize is said to have **overfit**. (We don't want overfitting!)

- Given data, how do we choose a prediction function?

e.g. 3 possible prediction function $H(\vec{x})$

  - $H_1(\vec{x}) = -2x_1 + 3x_5$
  - $H_2(\vec{x}) = \sin(x_1 \cdot x_2 \cdot x_3 \cdot x_4 \cdot x_5)$
  - $H_3(\vec{x}) = \sqrt{x_1 + x_3}(x_1 - x_2 x_5 + 100)$

General pipeline

- One common strategy is **empirical risk minimization** (ERM).
  - a.k.a., "minimizing expected loss"

Empirical Risk Minimization (ERM)

- Step 1: choose a **hypothesis class**
- Step 2: choose a **loss function**
- Step 3: minimize **expected loss (empirical risk)**

Outline

- Supervised Learning
  - Setting & Technical Terms
  - General pipeline

- Empirical Risk Minimization – Linear Regression
  - Step 1: Hypothesis class
  - Step 2: Loss function
  - Step 3: Minimize empirical risk

- Extension to Linear classification

ERM

- Step 1: choose a **hypothesis class**
- Step 2: choose a **loss function**
- Step 3: minimize **expected loss (empirical risk)**

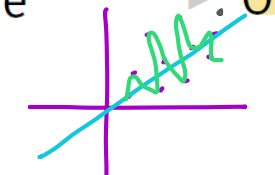
Hypothesis class

- A **hypothesis class** $\mathcal{H}$ is a set of possible prediction functions.

$H(\vec{x})$ ↑ prediction function

$\mathcal{H}$: a set of $H(\vec{x})$

- By choosing a hypothesis class, we are saying something about what the prediction function should look like.
- Examples:
  - $\mathcal{H}$:= linear functions (i.e. a set of linear function) $\{ \sum_{i=1}^{5} w_i x_i , \; w_i \in \mathbb{R} \}$
  - $\mathcal{H}$:= functions of the form $\sin(w_1 x_1 + \ldots x_5 x_5)$ $\{ \sin(w_1 x_1 + \cdots + w_5 x_5), \; w_i \in \mathbb{R} \}$ (w)
  - $\mathcal{H}$:= decision trees of depth 10
  - $\mathcal{H}$:= neural networks with one layer
- The more complex the hypothesis class, the greater the danger of **overfitting**.
  - Think: polynomials of degree 10 versus 2.



- Occam's Razor: assume *H* is simple.

degree n : $n >> 1$

degree 1

ERM – Step 1

A Simple Prediction Function

- We can go a long way by assuming our prediction functions to be **linear**.

  (linear w.r.t. parameter w_i)

$$H(\vec{x}) = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_3 + w_4 x_4 + w_5 x_5$$

  - This is a **linear prediction function**.
  - $w_0, w_1, \ldots, w_5$ are the "**parameters**" or "**weights**".
  - $\vec{w} = (w_0, \ldots, w_5)^T$ is a **parameter vector**.

ERM – Step 1

- There are infinitely many functions of the form

$$H(\vec{x}) = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_3 + w_4 x_4 + w_5 x_5 \quad , \quad w_i \in \mathbb{R}$$

- Each one is completely determined by $\vec{w}$.
  - Sometimes write $H(\vec{x}; \vec{w})$

input, parameter

e.g. $\vec{w} = \begin{bmatrix} -1 \\ 1 \\ 2 \\ 0 \\ 3 \\ 5 \end{bmatrix} \rightarrow H(\vec{x}) = -1 + x_1 + 2x_2 + 3x_4 + 5x_5$

"Parameterization"

(use $\vec{w}$ to parameterize function)

- Searching all linear functions ≡ searching over $\vec{w} \in \mathbb{R}^6$

- If there are d features, there are $d + 1$ parameters:

In General

bias (constant)

$$H(\vec{x}) = w_0 + w_1 x_1 + w_2 x_2 + \ldots + w_d x_d$$

$$= w_0 + \sum_{i=1}^{d} w_i x_i \quad \text{※}$$

ERM – Step 1

- The **augmented feature vector** $\text{Aug}(\vec{x})$ is the vector obtained by adding a 1 to the front of $\vec{x}$:

$$\vec{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{pmatrix} \in \mathbb{R}^d \qquad \text{Aug}(\vec{x}) = \begin{pmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_d \end{pmatrix} \in \mathbb{R}^{d+1}$$

- With augmentation, we can write as dot product:

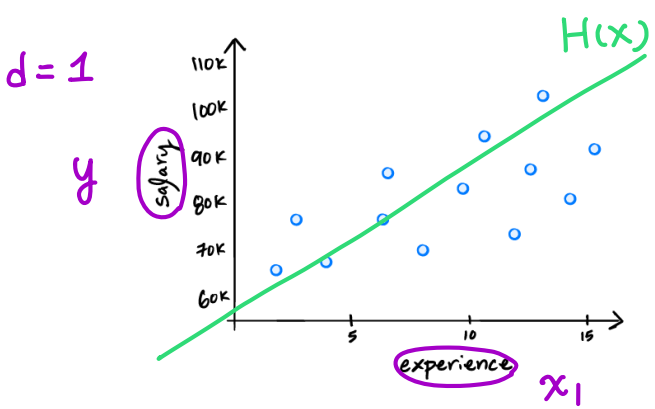
$$\begin{aligned} H(\vec{x}) &= w_0 + w_1 x_1 + w_2 x_2 + \ldots + w_d x_d \\ &= \text{Aug}(\vec{x}) \cdot \vec{w} \end{aligned}$$

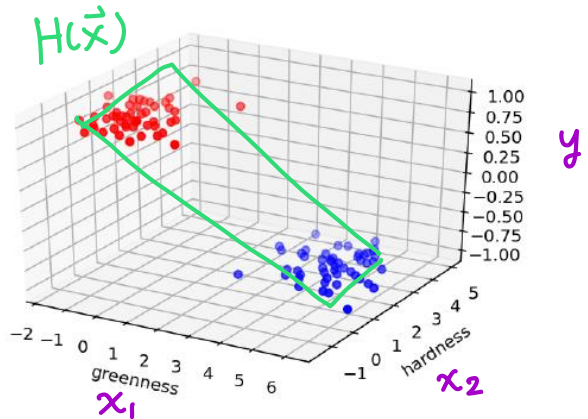
ERM – Step 1

Linear prediction function:
$H(\vec{x}) = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d$

- The **surface** of a prediction function H is the surface made by plotting $H(\vec{x})$ for all $\vec{x}$.

- If H is a linear prediction function, and[4]
  - $\vec{x} \in R^1$, then $H(x)$ is a straight line. → $d = 1$, $H(\vec{x}) = w_0 + w_1 x_1$ → line
  - $\vec{x} \in \mathbb{R}^2$, the surface is a plane. → $d = 2$, $H(\vec{x}) = w_0 + w_1 x_1 + w_2 x_2$ → hyperplane (2 dimensional)
  - $\vec{x} \in \mathbb{R}^d$, the surface is a d-dimensional **hyperplane**. → $H(\vec{x}) = w_0 + w_1 x_1 + \cdots + w_d x_d$ (d dimensional)





ERM – Step 2

(finished)

- Step 1: choose a **hypothesis class** : Choose linear prediction function set $\mathcal{H}$

$$H(\vec{x}) = Aug(\vec{x}) \cdot \vec{w} = w_0 + w_1 x_1 + \cdots + w_d x_d$$

(Now)

- Step 2: choose a **loss function**

- Step 3: minimize **expected loss (empirical risk)**

ERM – Step 2, choosing loss function

- Suppose we assume prediction function is linear.
  - There are still infinitely-many possibilities.
  - We'll pick one that works well on training data. → Need to define some metric to say "good" or "bad" for a given $H(\vec{x})$

$L(\cdot, \cdot) \to \mathbb{R}^+$ { high: very wrong; low: prediction is good

- A **loss function** quantifies how wrong a single prediction is.

$L(\text{prediction for example } i, \text{correct answer for example } i)$

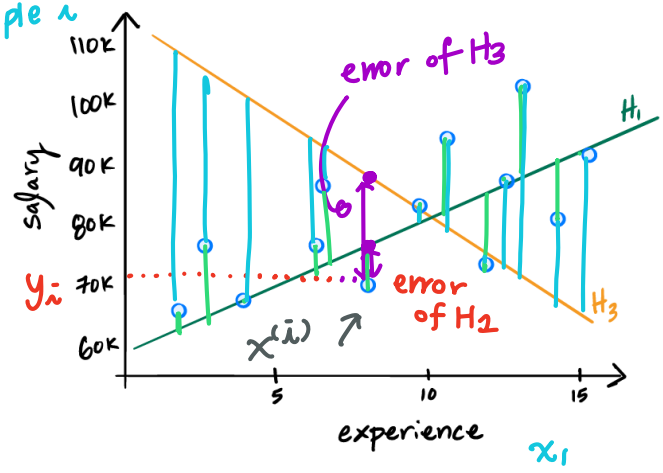
$L(H(\vec{x}^{(i)}), y_i)$ — ground-truth (label), correct answer for example i; prediction of example i

(0 loss is the best, no wrong prediction)

- We want $H(\vec{x}^{(i)}) \approx y_i$.

L:

1. **Absolute loss**: $|H(\vec{x}^{(i)}) - y_i|$
2. **Square loss**: $(H(\vec{x}^{(i)}) - y_i)^2$



ERM – Step 2, choosing loss function

- A good H is good *on average* over entire data set.
- The **expected loss** (or **empirical risk**) is one way of measuring this:

average loss across data points $\{x^{(i)}, y_i\}$

Risk

$$R(H) = \frac{1}{n}\sum_{i=1}^{n} L(H(\vec{x}^{(i)}), y_i)$$, n data points

- **Expected square loss** (mean squared error):

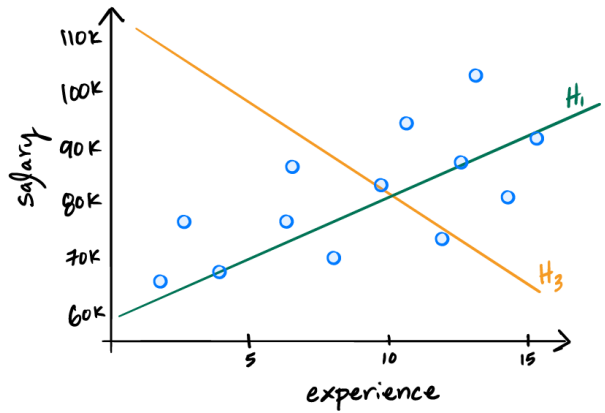
(L = square loss)

$$R_{sq}(H) = \frac{1}{n}\sum_{i=1}^{n} (H(\vec{x}^{(i)}) - y_i)^2$$

error

- This is the empirical risk for the square loss.

Empirical Risk



ERM – Step 3

(finished)
- Step 1: choose a **hypothesis class** $H(\vec{x})$ is linear prediction function)

(finished)
- Step 2: choose a **loss function** L : squared loss, $R_{sq}(H) = \frac{1}{n}\sum_{i=1}^{n}(H(\vec{x}^{(i)}) - y_i)^2$

(Next)
- Step 3: minimize **expected loss (empirical risk)**

how to minimize

ERM – Step 3, minimize empirical risk

- Goal: find H minimizing MSE.
- We want to find an H minimizing this:

$$R_{\text{sq}}(H) = \frac{1}{n}\sum_{i=1}^{n}(H(\vec{x}^{(i)}) - y_i)^2$$

(general form for squared loss: $H(\vec{x}^{(i)})$)

plug in "linear prediction function":

$H(\vec{x}) = \vec{w} \cdot \text{Aug}(\vec{x})$

- We want to find $\vec{w}$ that minimizes the average square loss:

$$R_{\text{sq}}(\vec{w}) = \frac{1}{n}\sum_{i=1}^{n}(\vec{w} \cdot \text{Aug}(\vec{x}^{(i)}) - y_i)^2$$

(squared loss for linear prediction function)

How to solve it?

we want to solve $\vec{w}$:

$\vec{w}^* \in \arg\min_{\vec{w}} R_{\text{sq}}(\vec{w})$

ERM – Step 3, minimize empirical risk

There is a closed-form solution!

"$R_{sq}(\vec{w}) = \frac{1}{n}\sum_{i=1}^{n}(\vec{w}\cdot \text{Aug}(\vec{x}^{(i)}) - y_i)^2$"

Use calculus: $\nabla_w R_{sq}(\vec{w})$

- Take the gradient, set to $\vec{0}$, solve.
- Solution: the **Normal Equations**, $\vec{w}^* = (X^T X)^{-1} X^T \vec{y}$
  - X is the **design matrix**

capital X

(c.f. $\vec{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_d \end{bmatrix}$)

features $x_1 \sim x_d$

$$X = \begin{pmatrix} \text{Aug}(\vec{x}^{(1)}) \longrightarrow \\ \text{Aug}(\vec{x}^{(2)}) \longrightarrow \\ \vdots \\ \text{Aug}(\vec{x}^{(n)}) \longrightarrow \end{pmatrix} = \begin{pmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \dots & x_d^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & \dots & x_d^{(2)} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_1^{(n)} & x_2^{(n)} & \dots & x_d^{(n)} \end{pmatrix} \leftarrow \text{data point } i$$

ERM – Step 3, minimize empirical risk

Note:

- There was a closed-form solution!
- This is a direct consequence of using the **mean squared error**. (and linear prediction function $H(\vec{x})$)
- Not true if we use, e.g., the **mean absolute error**.

Square loss: $(H(\vec{x}^{(i)}) - y_i)^2$

Absolute loss: $|H(\vec{x}^{(i)}) - y_i|$

Why linear? $H(\vec{x})$

- Easy to work with mathematically.
- Harder to overfit.
- But still quite powerful.

Outline

- Supervised Learning
  - Setting & Technical Terms
  - General pipeline
- Empirical Risk Minimization – Linear Regression
  - Step 1: Hypothesis class
  - Step 2: Loss function
  - Step 3: Minimize empirical risk
- Extension to Linear classification

Linear classification

Regression Task:

6.5, 5.3, 10, 0.2...

Given your friends movie rating (0-10) to predict "your rating of the movie"

Classification Task:

Given your friends movie rating (0-10) to predict "will you like the movie" (Y/N)

{1, 0}

- Linear prediction functions can be used in classification, too.

$$H(\vec{x}) = w_0 + w_1 x_1 + w_2 x_2 + \ldots + w_d x_d$$

- Same ERM paradigm also useful.

Linear classification

Build a classifier from a regressor:

- Binary classification can be thought of as regression where the targets are 1 and -1
  - (or 0 and 1, or ...)

(rating: 0~10) → like / not like: 5

6.5, 5.3, 10, 0.2 ...

Yes Yes Yes No

1 1 1 0

- $H(\vec{x})$ outputs a real number. Use the **sign** function to turn it into −1, 1:

(if we use)

Yes, No

1 −1

Yes: 1, No: −1

threshold: 0

$$\text{sign}(z) = \begin{cases} 1 & z > 0 \\ -1 & z < 0 \\ 0 & \text{otherwise} \end{cases}$$

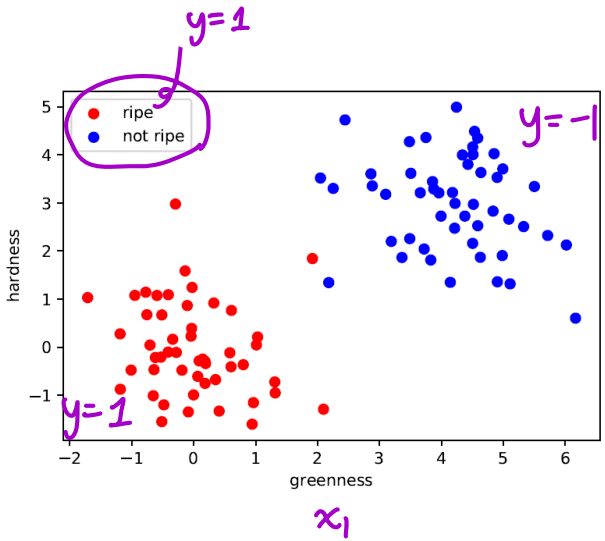
- Final prediction: $\text{sign}(H(\vec{x}))$

Linear classification

Another example: **Mango Ripeness**

- Predict whether a mango is ripe given greenness and hardness.
- Idea: gather a set of labeled **training data**.
  - Inputs along with correct output (i.e., "the answer").

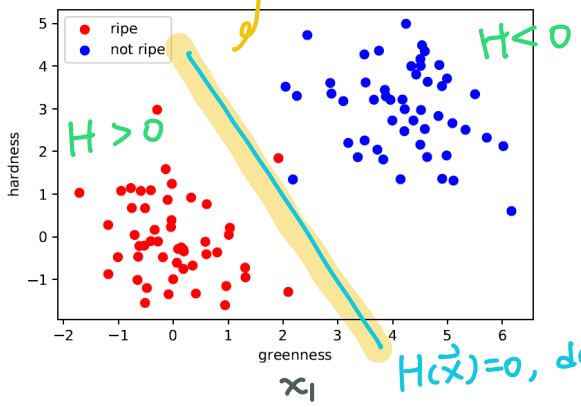
Greenness	Hardness	Ripe
0.7	0.9	1
0.2	0.5	-1
0.3	0.1	-1
⋮	⋮	⋮



Linear classification

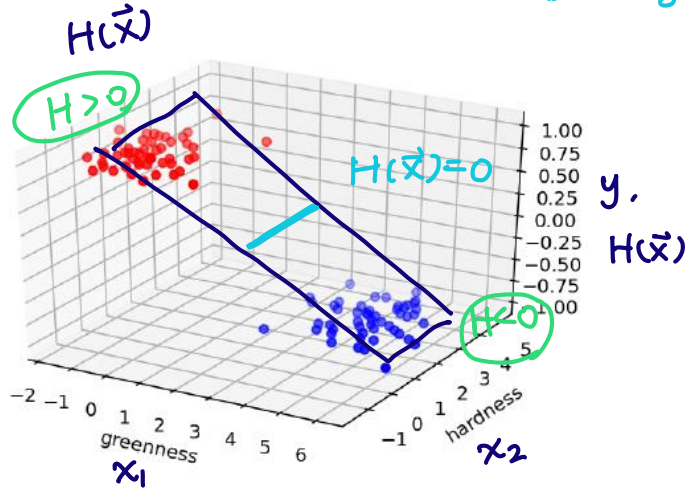
$x_2 = -\frac{w_1}{w_2} x_1 - \frac{w_0}{w_2}$

- The **decision boundary** is the place where the output of $H(x)$ switches from "yes" to "no".
  - If $H > 0 \mapsto$ "yes" and $H < 0 \mapsto$ "no", the decision boundary is where $H = 0$.

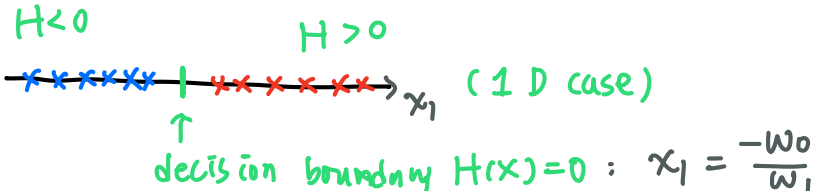


- If H is a linear predictor and[5] $w_0 + w_1 x_1 = 0 \rightarrow x_1 = \frac{-w_0}{w_1}$
  - $\vec{x} \in R^1$, then the decision boundary is just a number.
  - $\vec{x} \in \mathbb{R}^2$, the boundary is a straight line. $w_0 + w_1 x_1 + w_2 x_2 = 0$
  - $\vec{x} \in \mathbb{R}^d$, the boundary is a $d-1$ dimensional (hyper) plane.

$x_2 = \frac{-w_0}{w_2} + \frac{-w_1}{w_2} x_1$



[5] when plotted in the original feature coordinate space!



Linear classification

Least Squares and Outliers

(squared loss)

: caution : when using MSE, can be affected by outliers

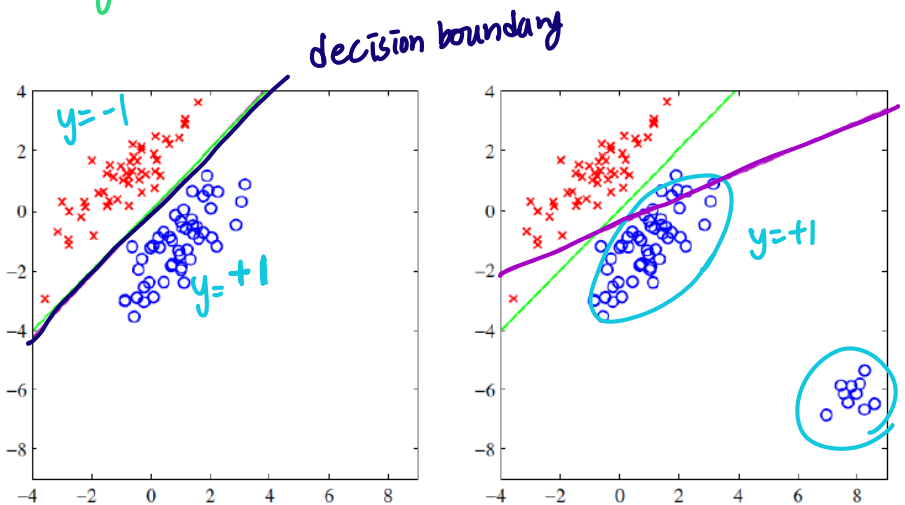


Figure 4.4 The left plot shows data from two classes, denoted by red crosses and blue circles, together with the decision boundary found by least squares (magenta curve) and also by the logistic regression model (green curve), which is discussed later in Section 4.3.2. The right-hand plot shows the corresponding results obtained when extra data points are added at the bottom left of the diagram, showing that least squares is highly sensitive to outliers, unlike logistic regression.

Supervised Learning

Feature Maps

Outline

- Motivating Example
- Basis function and feature map
  - General pipeline
  - Another example
- Visualization

Outline

- Motivating Example
- Basis function and feature map
  - General pipeline
  - Another example
- Visualization

Motivating Example: Last Lecture

$H(\vec{x}) = w_0 + w_1 x_1 + w_2 x_2$. goal: solve for w_0, w_1, w_2 $(\vec{w})$. given $\{x^{(i)}, y_i\}_{i=1}^n$

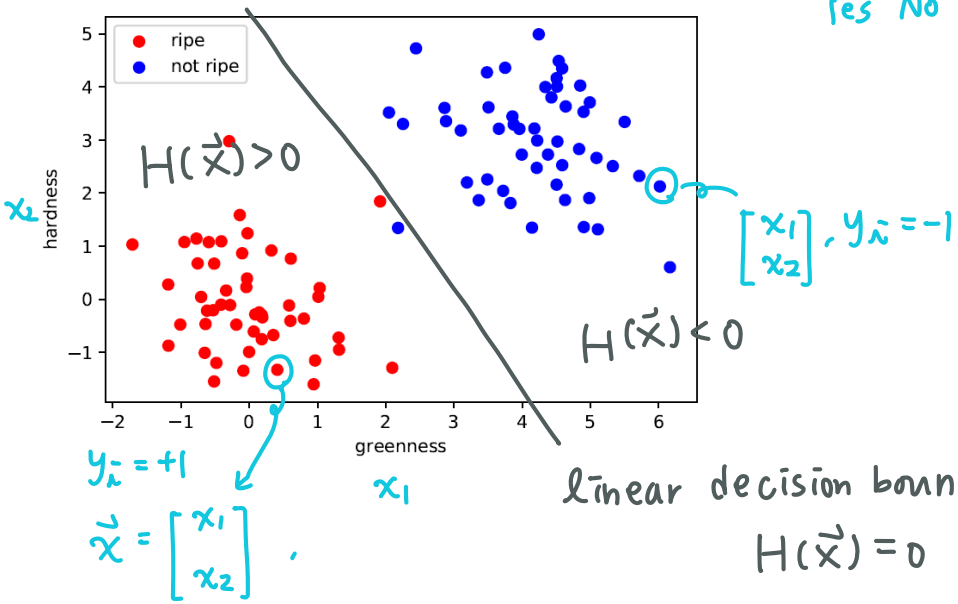
Task:

Predict Mango Ripeness

$\{x^{(i)}, y_i\}_{i=1}^n$

$\{+1, -1\}$

Yes No



$$H(\vec{x}; \vec{w}) = w_0 + w_1 x_1 + \ldots + w_d x_d$$
$$= \text{Aug}(\vec{x}) \cdot \vec{w}$$

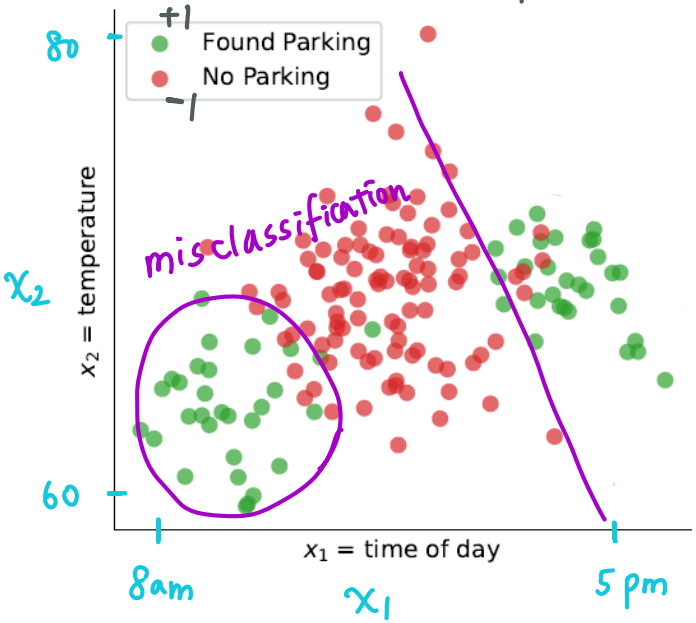
Motivating Example: This Lecture

Goal: find $H(\vec{x})$ to predict parking availability accurately

Task:

Predict Parking Availability

Classification problem: $y = \{+1, -1\}$



Imagine a scatter plot of the training data with the two features:

- x_1 = time of day
- x_2 = temperature

"yes" examples are green, "no" are red.

$H(\vec{x}; \vec{w})$

Can we still use linear function: No.

$$H(\vec{x}; \vec{w}) = w_0 + w_1 x_1 + \ldots + w_d x_d$$

What can we do ?

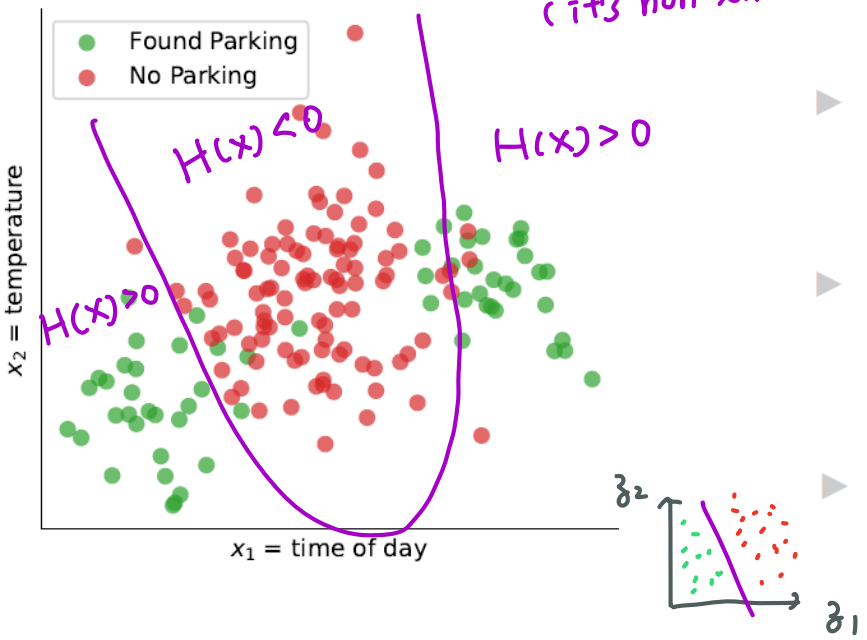
Can we still use linear prediction function:

$H(\vec{z}; \vec{w})$, it's linear $\vec{z}$, $\vec{z}$ = transformed variable

= non-linear features

Motivating Example: This Lecture

Task:
Predict Parking Availability

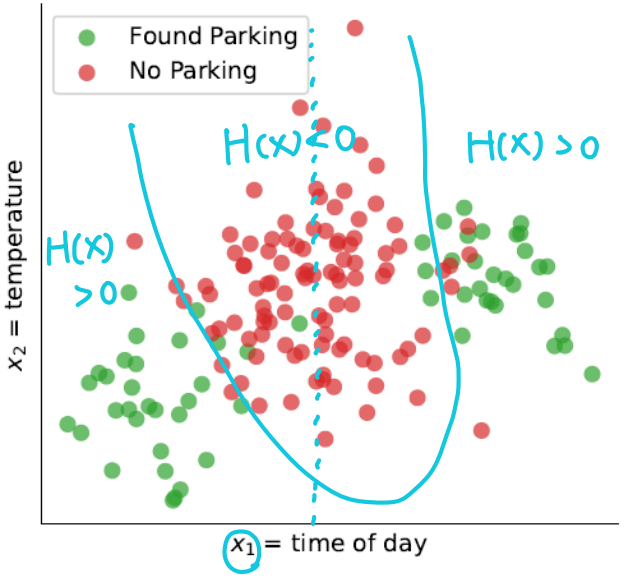


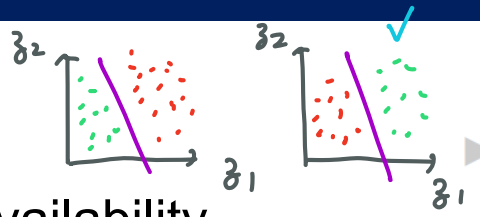
- How do we learn non-linear patterns using linear prediction functions?
- We **represented** the data with two features: time and temperature
- In this **representation**, the trend is **nonlinear**.
  - There is no good linear decision function
  - Learning is "difficult".
- **Idea**: We'll make a new **representation** by creating **new features** from the **old features**.

z_1, z_2 x_1, x_2

Motivating Example: This Lecture

Task:
Predict Parking Availability





- What new features should we create?
  - The "right" representation makes the problem easy again.
  - Linear prediction functions[1] work well when relationship is linear
    - When x is small we should predict -1
    - When x is large we should predict +1

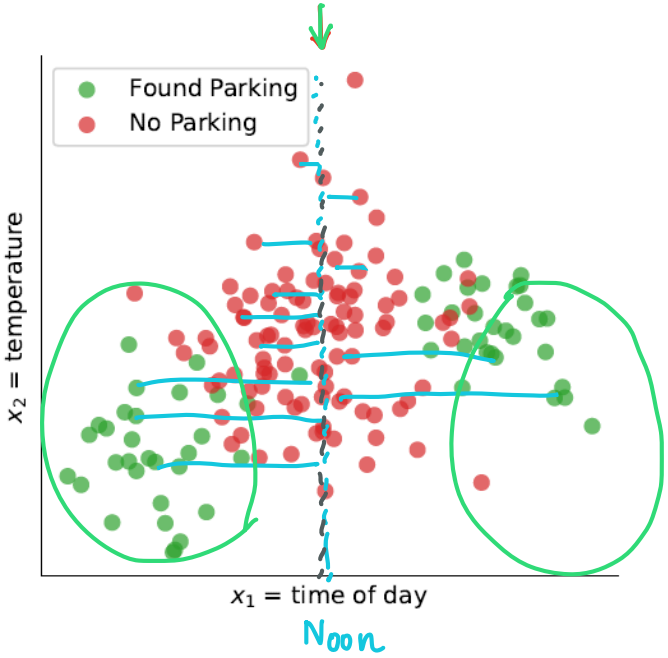
How can we "transform" the time of day x_1 to create a new feature x_1' satisfying:

or z_1

- When x_1' is small, we should predict -1
- When x_1' is large, we should predict +1

Motivating Example: This Lecture

Task:
Predict Parking Availability

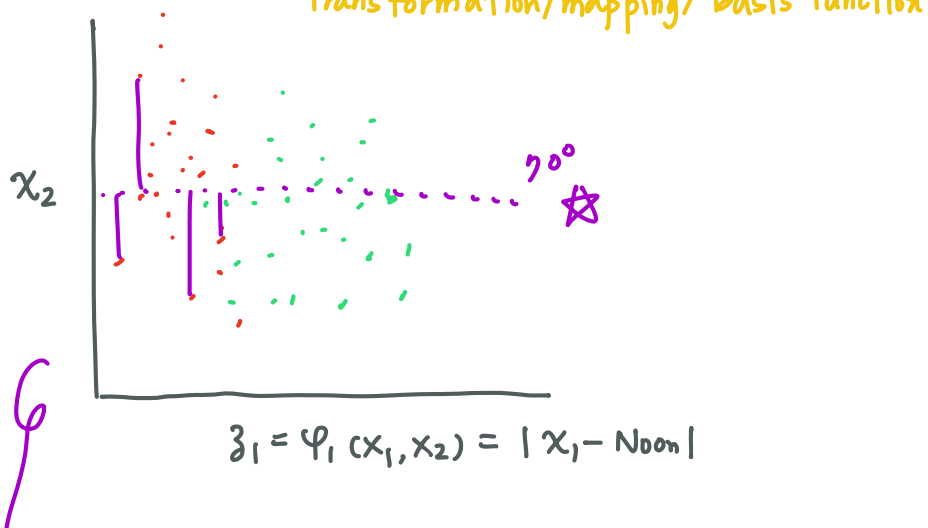


Idea

- Transform "time" to "absolute time until/since Noon"

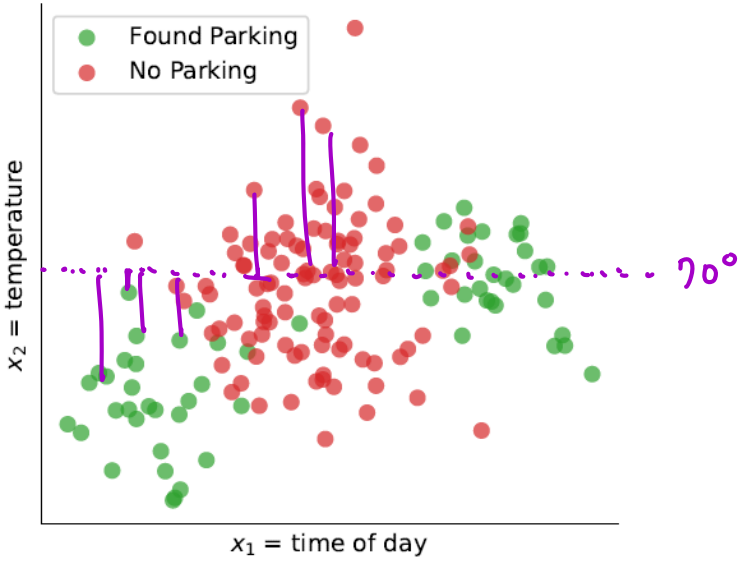
$z_1 = \varphi_1(x_1, x_2) = |x_1 - \text{Noon}|$

transformation/mapping/basis function



Motivating Example: This Lecture

Task:
Predict Parking Availability



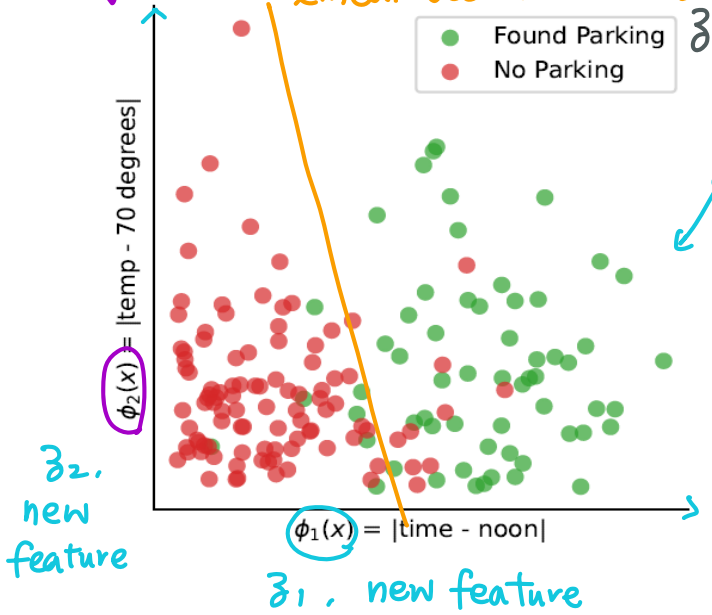
transform x_2 to z_2

Additionally,

- Transform "temp." to "absolute difference between temp. and 70°"

now: can use linear decision boundary (in z)

$z_2 = \varphi_2(x_1, x_2) = |x_2 - 70°|$



New data representation

z_2, new feature

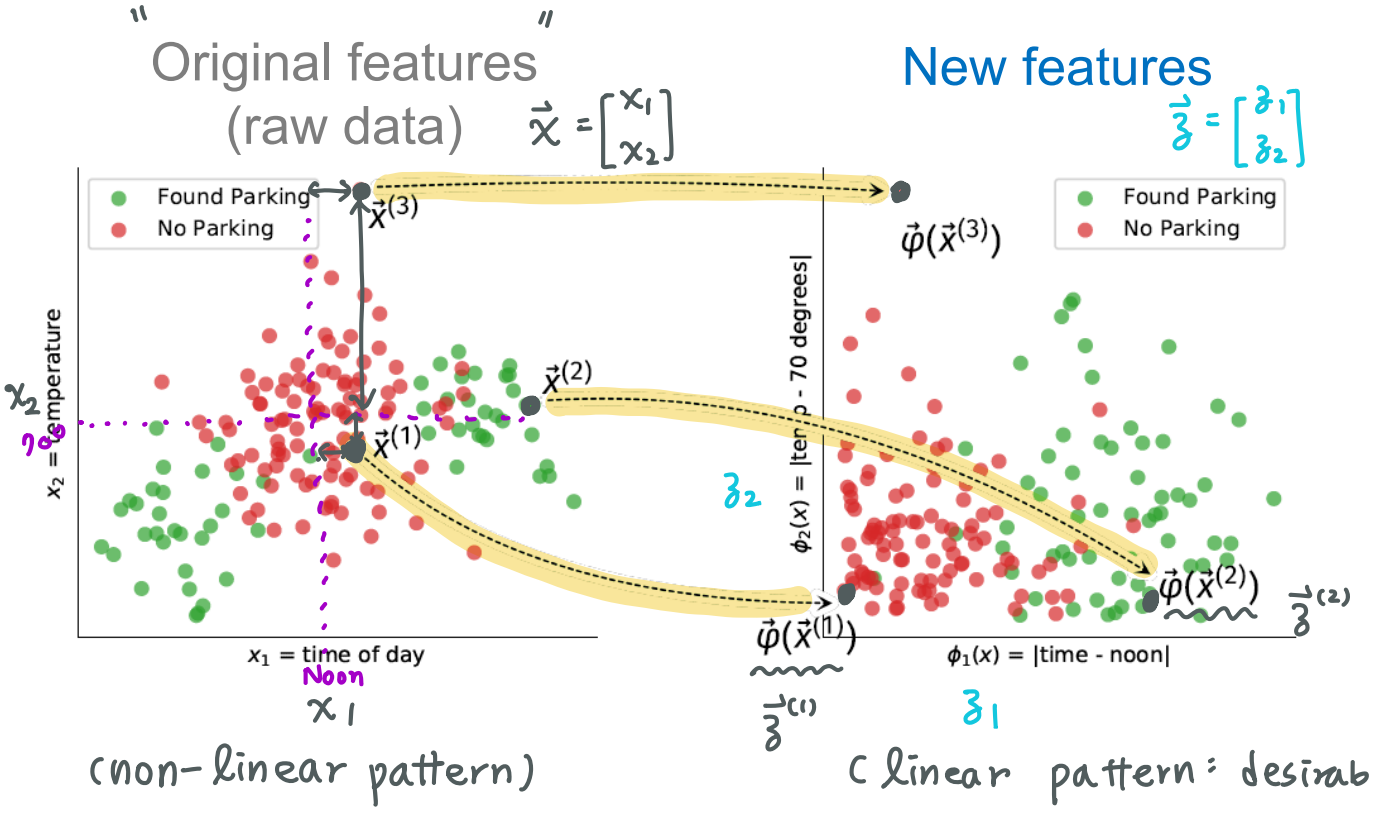
z_1, new feature

The new data representation z_1, z_2 is good because (φ_1) (φ_2) now we can use linear decision boundary!

Outline

- Motivating Example
- Basis function and feature map
  - General pipeline
  - Another example
- Visualization

Basis function & feature maps



$\mathbb{R}^d \to \mathbb{R}$

non-linear **basis functions**:

$$\varphi_1(x_1, x_2) = |x_1 - \text{Noon}|$$
$$\varphi_2(x_1, x_2) = |x_2 - 70^\circ|$$

Define $\vec{\varphi}(\vec{x}) = (\varphi_1(\vec{x}), \varphi_2(\vec{x}))^T$

$\vec{\varphi}$ is a **feature map**

Basis function & feature maps

- The basis functions φ_1, φ_2 give us our "new" features.

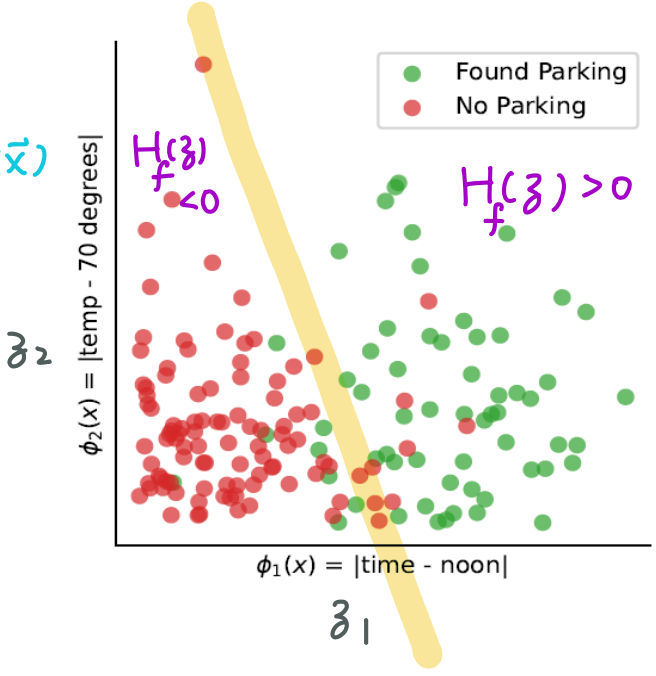
$$z_i = \varphi_i(\vec{x}) \qquad z = \begin{bmatrix} z_1 \\ \vdots \\ z_d \end{bmatrix} = \begin{bmatrix} \varphi_1(\vec{x}) \\ \vdots \\ \varphi_d(\vec{x}) \end{bmatrix} = \varphi(\vec{x})$$

- This gives us a new **representation**. z_i
  - Map each training example $\vec{x}^{(i)}$ to feature space, creating new training data:

$$\vec{z}^{(1)} = \vec{\varphi}(\vec{x}^{(1)}), \quad \vec{z}^{(2)} = \vec{\varphi}(\vec{x}^{(2)}), \quad \ldots, \quad \vec{z}^{(n)} = \vec{\varphi}(\vec{x}^{(n)})$$

- In this representation, learning (classification) is easier.
  - Fit linear prediction function H in usual way:

$$H_f(\vec{z}) = W_0 + W_1 z_1 + W_2 z_2 + \ldots + W_d z_d$$



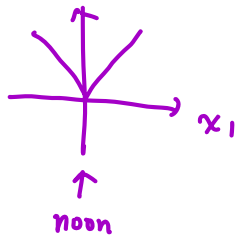
$H(\vec{x})$ (in the original space)

$H_f(\vec{z})$ (in the new/feature space)

General pipeline: Feature mapping

- Q: How do we learn non-linear patterns using linear prediction functions?
- A: Use non-linear basis functions to map to a feature space.

$\varphi_1(x_1, x_2) = |x_1 - \text{noon}|$



1. Start with data in original space, $\mathbb{R}^d$. $\vec{x} \in \mathbb{R}^d$
2. Choose some basis functions, $\varphi_1, \varphi_2, \ldots, \varphi_{d'}$ (d' can be different from d)
3. Map each data point to **feature space** $\mathbb{R}^{d'}$:

$$\vec{x} \mapsto (\varphi_1(\vec{x}), \varphi_2(\vec{x}), \ldots, \varphi_{d'}(\vec{x}))^T = [z_1, z_2, \cdots, z_{d'}]^T$$

↑ new features

4. Fit linear prediction function in new space:

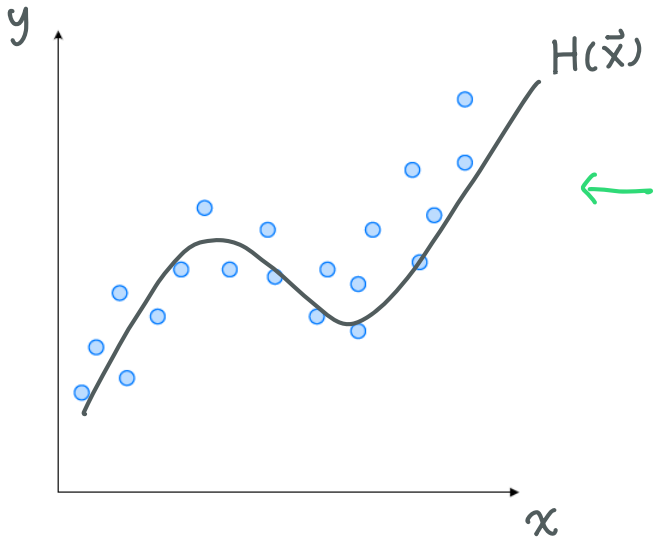
$H(\vec{x})$: Linear function of $\vec{w}$, non-linear function of $\vec{x}$.

$$H(\vec{x}) = w_0 + w_1\varphi_1(\vec{x}) + w_2\varphi_2(\vec{x}) + \cdots + w_{d'}\varphi_{d'}(\vec{x}),$$

Another example

(Regression problem: $y \in \mathbb{R}$)

How to fit this non-linear patterns?



How to find $H(\vec{x})$

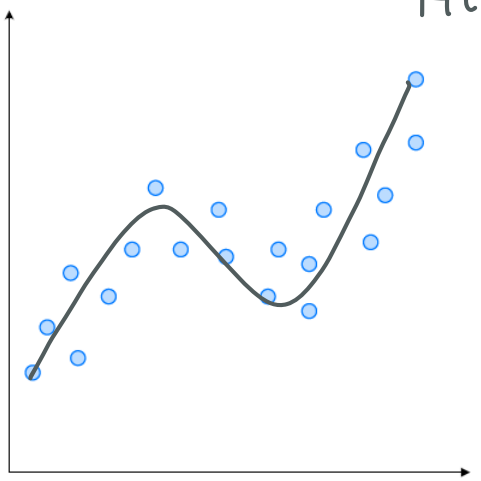
$$H(\vec{x}) = w_0 + w_1\varphi_1(\vec{x}) + w_2\varphi_2(\vec{x}) + \ldots + w_d\varphi_d(\vec{x})$$

The patterns look like polynomial:

⇒ choose the basis function function to be polynomial!

Another example

How to fit this non-linear patterns?



$H(\vec{x}) = \vec{w} \cdot \text{Aug}(\vec{z})$

$\vec{z} = \begin{bmatrix} z_1 \\ \vdots \\ z_4 \end{bmatrix} = \begin{bmatrix} \varphi_1(x) \\ \vdots \\ \varphi_4(x) \end{bmatrix}$

non-linear features $\varphi_1(x)\ \varphi_2(x)\ \varphi_3(x)\ \varphi_4(x)$

$$x \mapsto (x, x^2, x^3, x^4)$$

- Treat x, x^2, x^3, x^4 as **new** features.
- Create design matrix: $\varphi_1\ \varphi_2\ \varphi_3\ \varphi_4$ new features

$$X = \begin{pmatrix} 1 & x_1 & x_1^2 & x_1^3 & x_1^4 \\ 1 & x_2 & x_2^2 & x_2^3 & x_2^4 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 & x_n^3 & x_n^4 \end{pmatrix} \begin{matrix} \leftarrow \text{data 1 : } x_1 \\ \leftarrow \text{data 2 : } x_2 \\ \\ \leftarrow \text{data n : } x_n \end{matrix}$$

$$\vec{w} = (X^\top X)^{-1} X^\top \vec{y}$$

(Same form of solution, but need to use the above design matrix X !)

- Works for more than just polynomials.

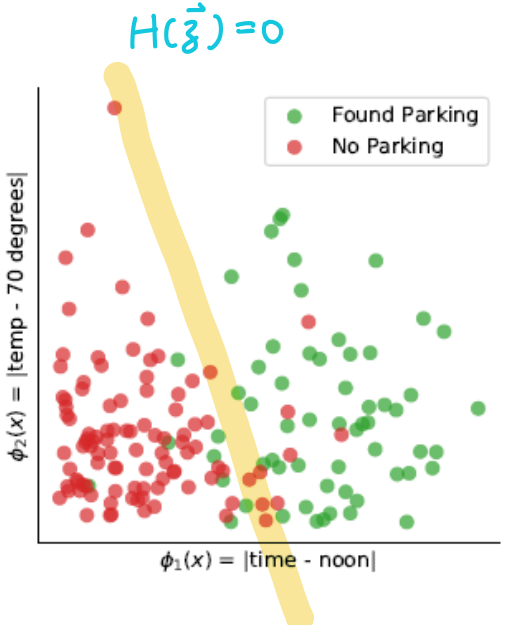
$\varphi_i(x)$ could be other function too

Outline

- Motivating Example
- Basis function and feature map
  - General pipeline
  - Another example
- Visualization

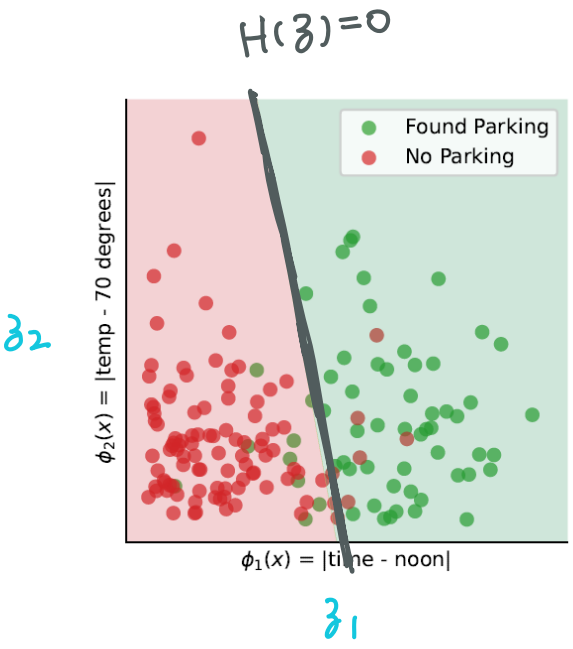
Visualization

- The **original space**: where the raw data lies. $\vec{x}$
- The **feature space**: where the data lies after feature mapping $\vec{\varphi}$: $\vec{z}$
- Remember: we fit a linear prediction function in the **feature space**.
  - In **feature space**, what does the decision boundary look like? $H(\vec{z}) = 0$
  - What does the prediction function surface look like? $H(\vec{z})$



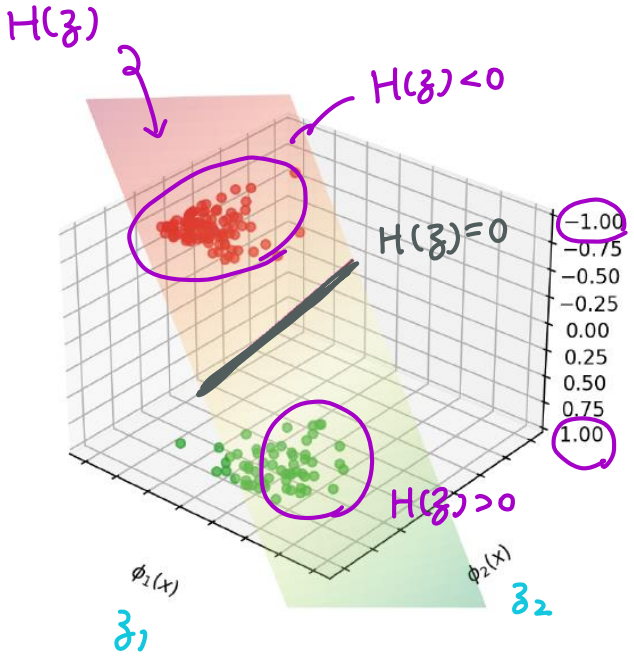
Visualization

Decision boundary in **Feature** space



(Fit by minimizing square loss)

Prediction Surface in **Feature** space



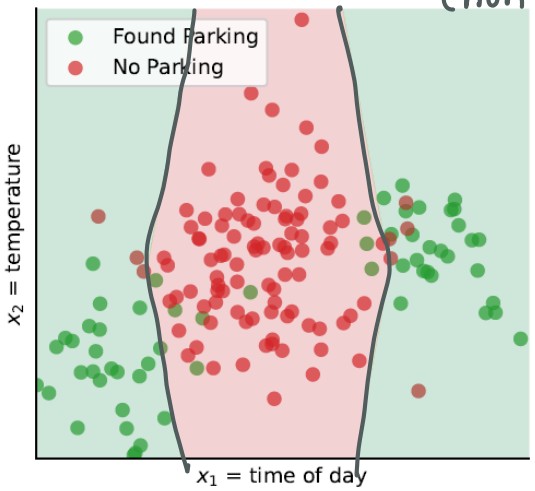
Visualization

Decision boundary in **Original** space

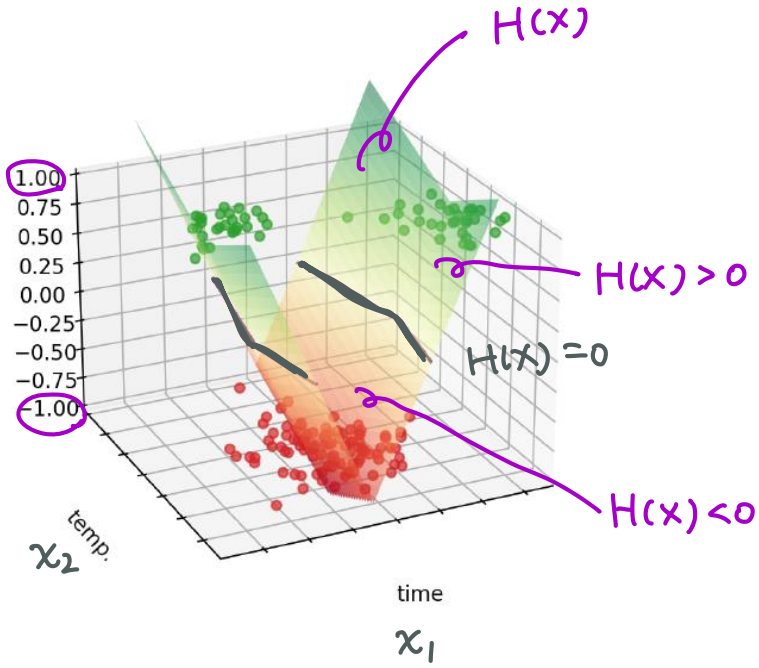
non linear

$H(\vec{x}) = w_0 + w_1 \varphi_1(x) + w_2 \varphi_2(x)$

$H(x) = 0$ decision boundary (non-linear!)



Prediction Surface in **Original** space



Supervised Learning

Radial Basis Function

Outline

- Feature mapping
  - Review
  - Gaussian Basis function
    - Examples
- Radial Basis function (RBF)
  - RBF network

Outline

- Feature mapping
  - Review
  - Gaussian Basis function
    - Examples
- Radial Basis function (RBF)
  - RBF network

Feature Mapping: Review

1. Start with data in original space, $\mathbb{R}^d$. $\vec{x} \in \mathbb{R}^d$
2. Choose some basis functions, $\varphi_1, \varphi_2, \ldots, \varphi_{d'}$ (d' can be different from d)
3. Map each data point to **feature space** $\mathbb{R}^{d'}$:

$$\vec{x} \mapsto (\varphi_1(\vec{x}), \varphi_2(\vec{x}), \ldots, \varphi_{d'}(\vec{x}))^T = [z_1, z_2, \cdots, z_{d'}]^T$$

$$\vec{\varphi} = \begin{bmatrix} \varphi_1(\vec{x}) \\ \varphi_2(\vec{x}) \\ \vdots \\ \varphi_{d'}(\vec{x}) \end{bmatrix}$$

↑ new features

4. Fit linear prediction function in new space:

$H(\vec{x})$: Linear function of $\vec{w}$, non-linear function of $\vec{x}$.

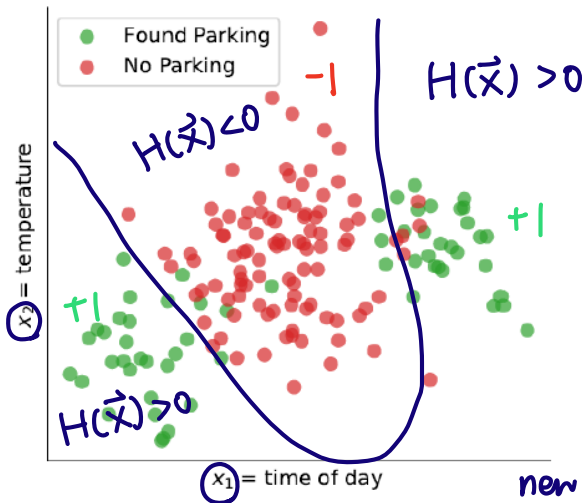
$$H(\vec{x}) = w_0 + w_1 \varphi_1(\vec{x}) + w_2 \varphi_2(\vec{x}) + \cdots + w_{d'} \varphi_{d'}(\vec{x}),$$

$\varphi_i(\vec{x})$

Feature Mapping: Review

$$H(\vec{x}) = w_0 + w_1\varphi_1(\vec{x}) + w_2\varphi_2(\vec{x}) + \ldots + w_d\varphi_d(\vec{x})$$

Linear classification task

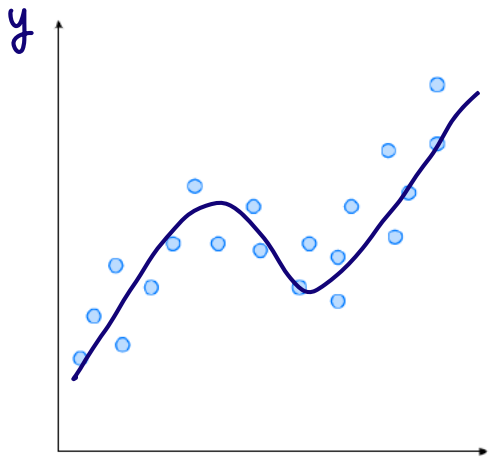


$z_1 = \varphi_1(\vec{x}) = |x_1 - \text{noon}|$

$z_2 = \varphi_2(\vec{x}) = |x_2 - 70^\circ|$

new feature $\vec{z} = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix}$, $H(\vec{z})$

Linear regression task



$z_1 = \varphi_1(\vec{x}) = x$

$z_2 = \varphi_2(\vec{x}) = x^2$

$z_3 = \varphi_3(\vec{x}) = x^3$, $z_4 = \varphi_4(\vec{x}) = x^4$

$H(\vec{z})$

new feature $\vec{z} = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \\ z_4 \end{bmatrix}$

Feature Mapping: Review

Generic Basis Functions

- The basis functions we used before were engineered using domain knowledge.
- They were specific to the problem at hand.
- **Very manual process!**
- **Now:** features that work for many problems.

Outline

- Feature mapping
  - Review
  - Gaussian Basis function
    - Examples
- Radial Basis function (RBF)
  - RBF network

Gaussian Basis Functions

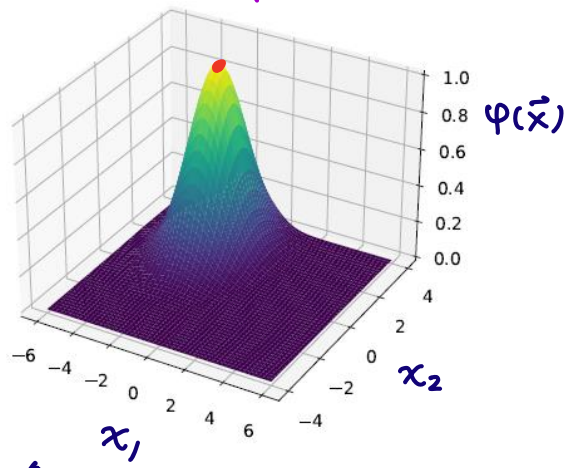
- A common choice: **Gaussian** basis functions:

$$\varphi(\vec{x};\vec{\mu},\sigma) = e^{-\|\vec{x}-\vec{\mu}\|^2/\sigma^2}$$

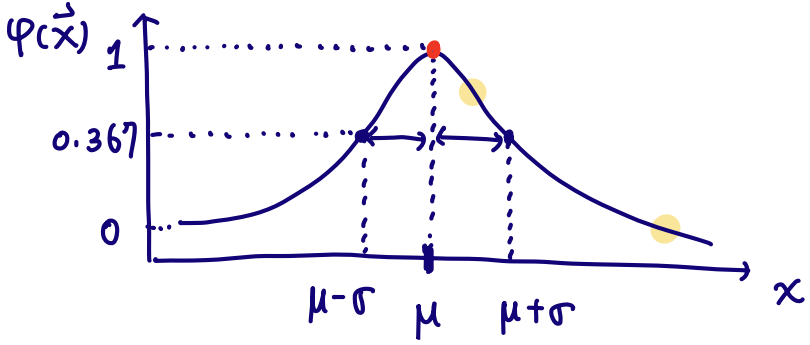
- $\vec{\mu}$ is the center. σ controls the "width"

$\vec{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$

2D case:



1D case: $\vec{x} = x$



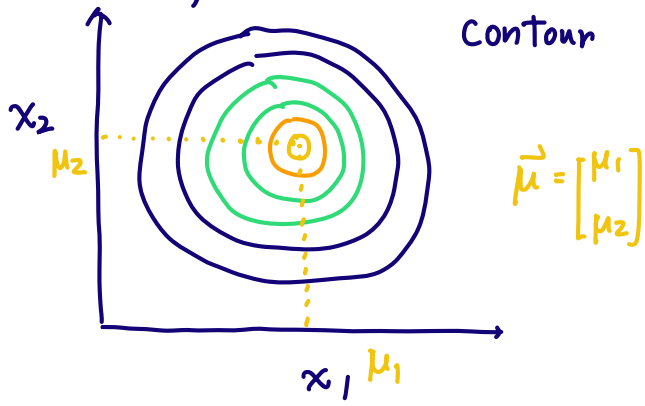
$\varphi(\vec{\mu}) = e^{-0} = 1$

$\varphi(\vec{\mu}\pm\sigma) = e^{-\sigma^2/\sigma^2}$

$= e^{-1} = 0.367$

$\|\vec{x}-\vec{\mu}\|^2 >> 1$, $e^{-\text{large positive \#}}$

$= e^{-\infty} \to 0$



Gaussian Basis Functions

- If $\vec{x}$ is close to $\vec{\mu}$, $\varphi(\vec{x}; \vec{\mu}, \sigma)$ is large.
  ($\vec{x}$ is similar to $\vec{\mu}$)
- If $\vec{x}$ is far from $\vec{\mu}$, $\varphi(\vec{x}; \vec{\mu}, \sigma)$ is small.

- Intuition: φ measures how "similar" $\vec{x}$ is to $\vec{\mu}$.
  - Assumes that "similar" objects have close feature vectors.

Gaussian Basis Functions: New representation

- Pick number of new features. d'.
  - Pick centers for Gaussians $\vec{\mu}^{(1)}, \ldots, \vec{\mu}^{(2)}, \ldots, \vec{\mu}^{(d')}$
  - Pick widths: $\sigma_1, \sigma_2, \ldots, \sigma_{d'}$ (usually all the same)
  - Define ith basis function: $\varphi_i(\vec{x}) = e^{-\|\vec{x}-\vec{\mu}^{(i)}\|^2/\sigma_i^2}$
- For any feature vector $\vec{x} \in \mathbb{R}^d$, map to vector $\vec{\varphi}(\vec{x}) \in \mathbb{R}^{d'}$.
  - φ_1: "similarity" of $\vec{x}$ to $\vec{\mu}^{(1)}$
  - φ_2: "similarity" of $\vec{x}$ to $\vec{\mu}^{(2)}$
  - ...
  - $\varphi_{d'}$: "similarity" of $\vec{x}$ to $\vec{\mu}^{(d')}$
- Train linear classifier in this new representation.
  - E.g., by minimizing expected square loss. → solve for w_i.

feature map $\varphi(\vec{x}) = \begin{bmatrix} \varphi_1(\vec{x}) \\ \vdots \\ \varphi_{d'}(\vec{x}) \end{bmatrix}$

new feature $\vec{z} = \varphi(\vec{x})$

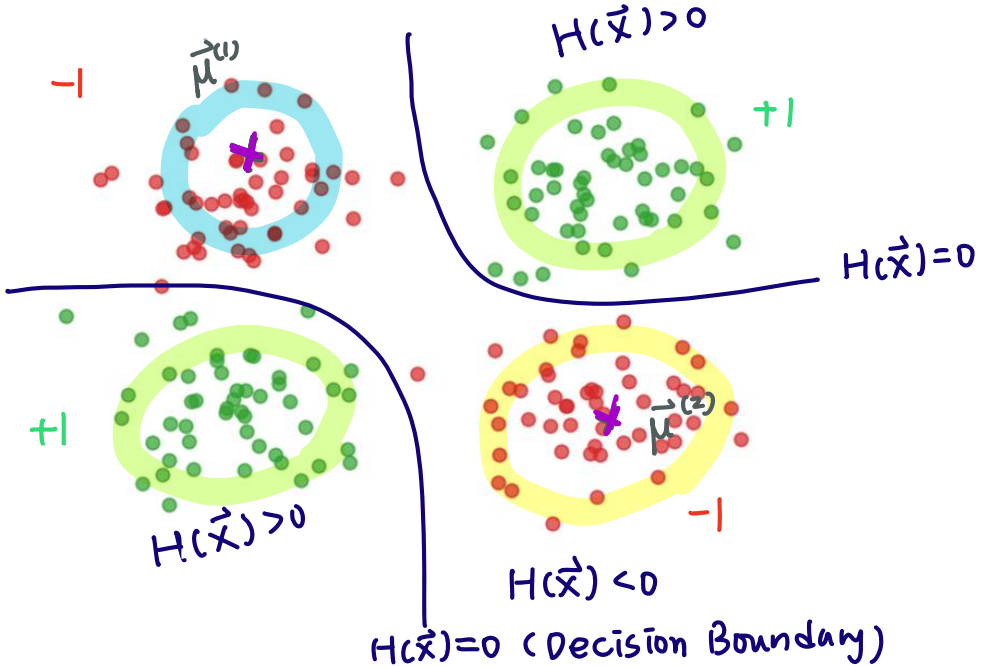
$H(\vec{z}) = w_0 + w_1 z_1 + \cdots + w_{d'} z_{d'}$

Example

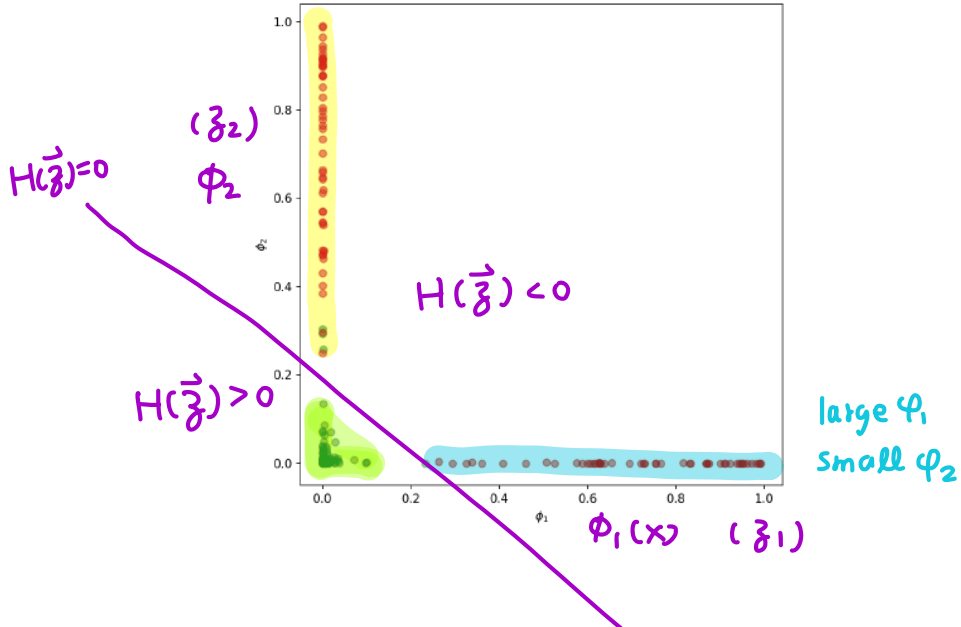
Ans: Need 2 Gaussian basis func : ϕ_1 & ϕ_2

How many Gaussian basis functions would you use, and where would you place them to create a new representation for this data?

goal: new feature → linear prediction!



small ϕ_1, large ϕ_2



Neural Networks: Multilayer Perceptron

Beyond RBFs

- When training RBFs, we fixed the basis functions *before* training the weights.
- Representation learning was decoupled from learning the prediction function.
- **Now:** learn representation **and** prediction function together.

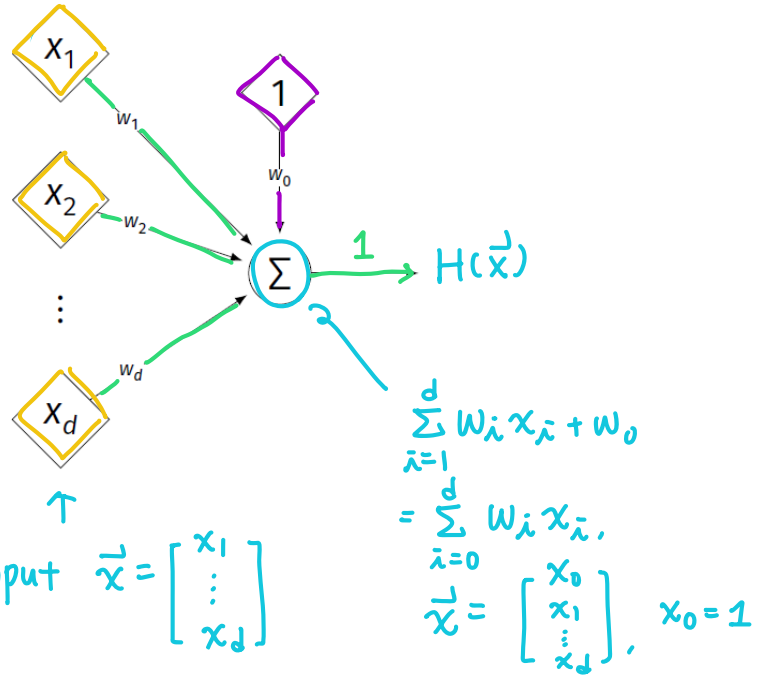
$$h_f(x) = w_0 + w_1 \varphi_1(\vec{x}) + w_2 \varphi_2(\vec{x}) + \cdots + w_k \varphi_k(\vec{x})$$

$$\varphi_i(\vec{x}) = e^{-\|\vec{x} - \vec{\mu}^{(i)}\|/\sigma_i^2}$$

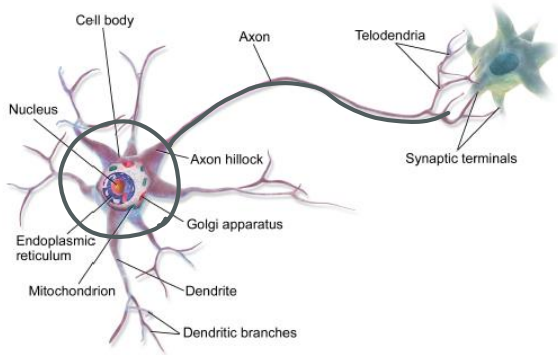
Linear Models

(bias term)

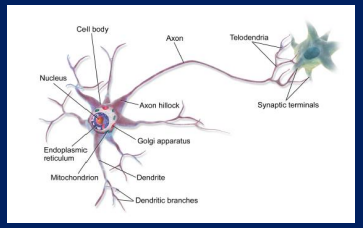
$$H(\vec{x}) = w_0 + w_1 x_1 + \ldots + w_d x_d$$



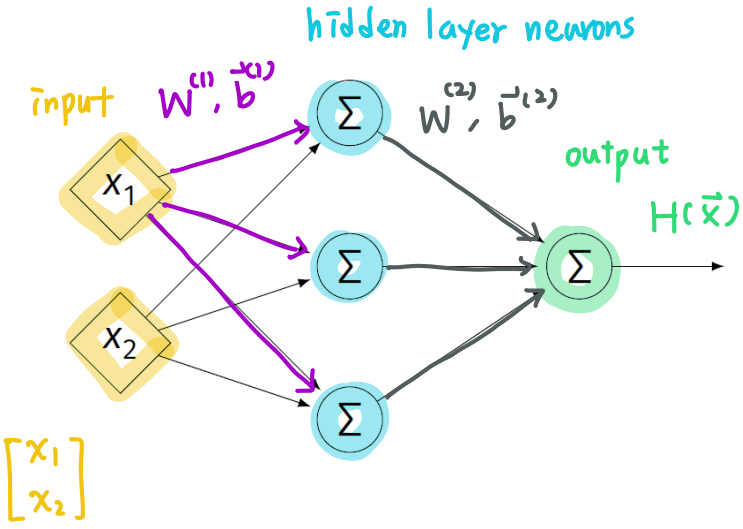
neuron



Neural Networks



- Neurons are organized into **layers**.
  - **Input layer**, **output layer**, and **hidden layers**.



$$W^{(1)} = \begin{pmatrix} 2 & -1 & 0 \\ 4 & 5 & 2 \end{pmatrix} \quad W^{(2)} = \begin{pmatrix} 3 \\ 2 \\ -4 \end{pmatrix}$$

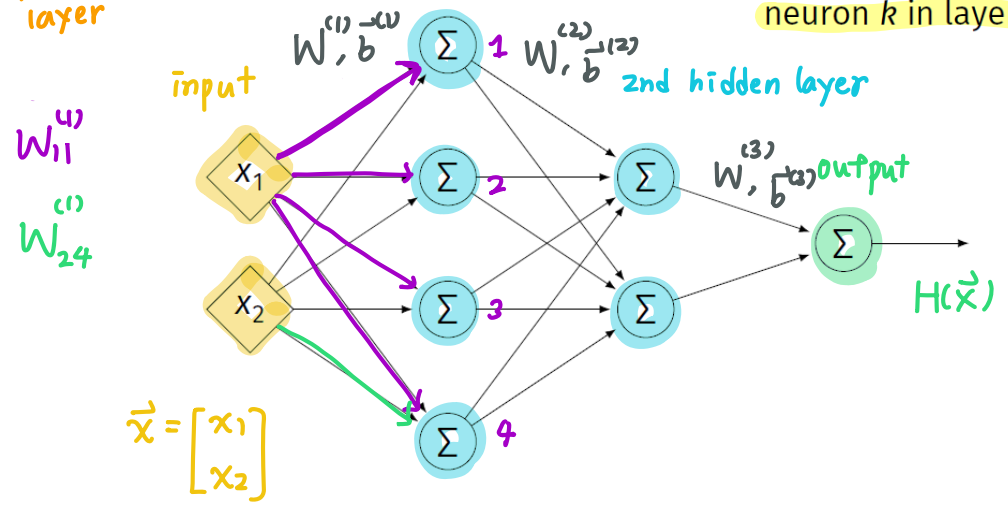
$$\vec{b}^{(1)} = (3, -2, -2)^T \quad \vec{b}^{(2)} = (-4)^T$$

$W_{jk}^{(i)} \leftarrow$ layer i

neuron index previous layer

neuron index in next layer

- $W_{jk}^{(i)}$ denotes weight of connection between neuron j in layer $(i-1)$ and neuron k in layer i



NN parameters

$$W^{(1)} = \begin{pmatrix} 2 & -1 & -3 & 0 \\ 4 & 5 & -7 & 2 \end{pmatrix} \quad W^{(2)} = \begin{pmatrix} 1 & 2 \\ -4 & 3 \\ -6 & -2 \\ 3 & 4 \end{pmatrix} \quad W^{(3)} = (-1 \quad 5)$$

$$\vec{b}^{(1)} = (3, 6, -2, -2)^T \quad \vec{b}^{(2)} = (-4, 0)^T \quad \vec{b}^{(3)} = (1)^T$$

Neural Networks

- Number of cells in input layer determined by dimensionality of input feature vectors.

$$\vec{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_d \end{bmatrix}$$

- Number of cells in hidden layer(s) is determined by you.
- Output layer can have >1 neuron.
- Can have more than one hidden layer.
  - A network is "**deep**" if it has >1 hidden layer.

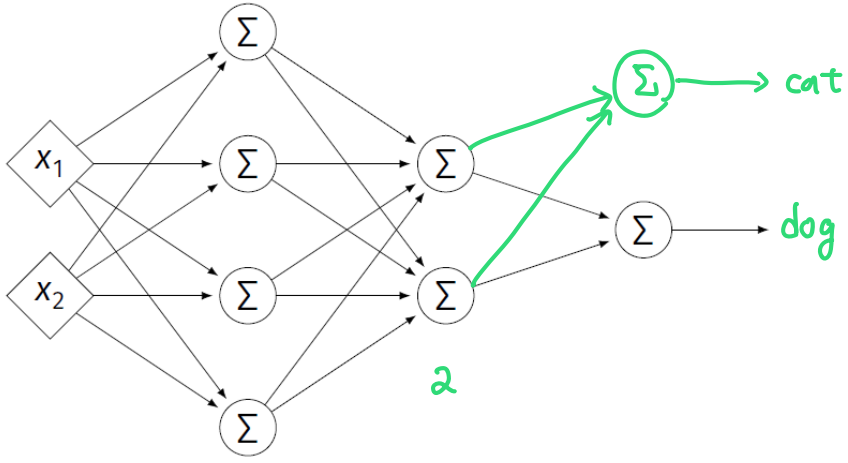
"depth" of networks: # of hidden layer

- Hidden layers can have different number of neurons.

e.g. 4 in 1st hidden layer, 2 in 2nd hidden layer

Multi-classification

e.g.
2 output neurons



Neural Networks

1st hidden layer: $\vec{z}^{(1)} \in \mathbb{R}^4$

$\vec{z}^{(1)} = W^{(1)T}\vec{z}^{(0)} + \vec{b}^{(1)}$

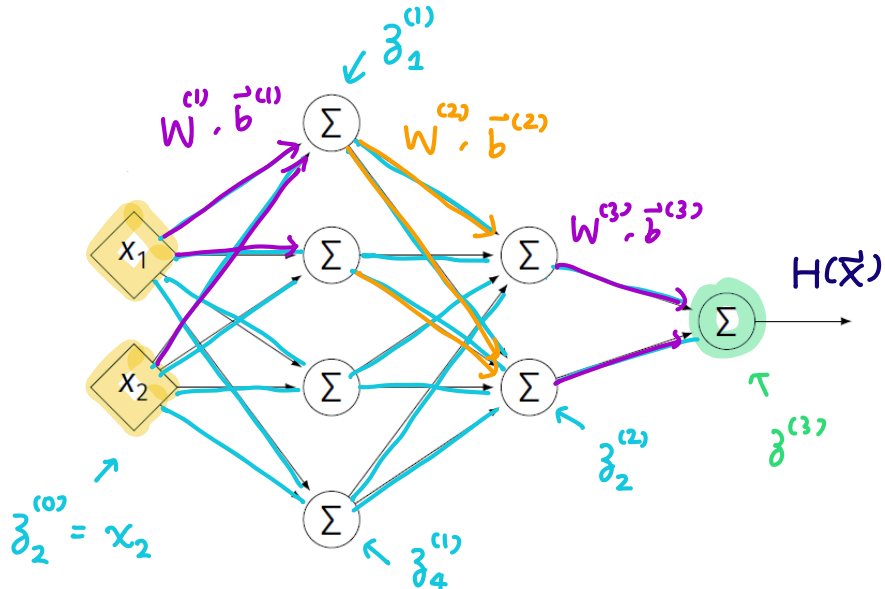
2nd hidden layer: $\vec{z}^{(2)} \in \mathbb{R}^2$

$\vec{z}^{(2)} = W^{(2)T}\vec{z}^{(1)} + \vec{b}^{(2)}$

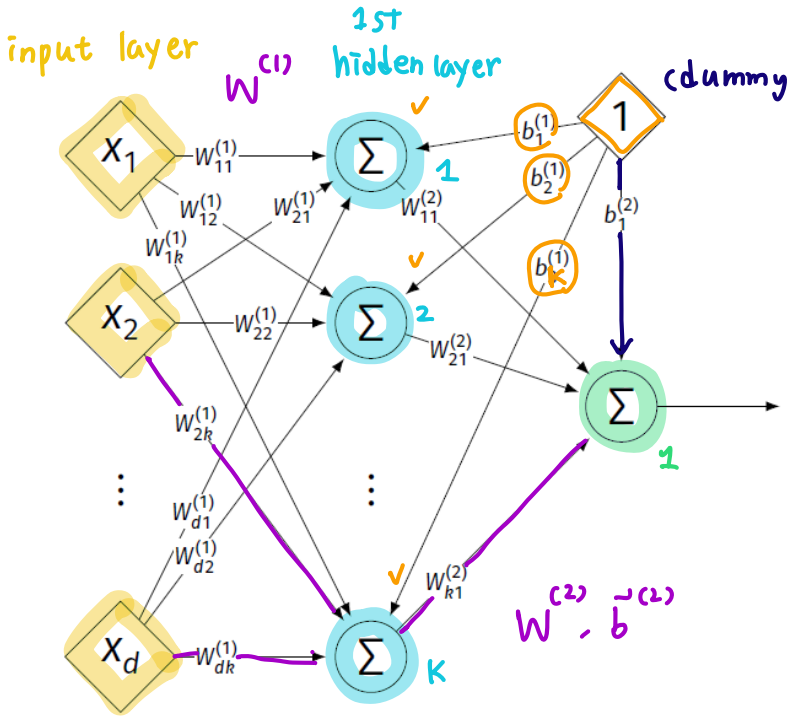
- These are "**fully-connected, feed-forward**" networks with one output.
- They are functions $H(\vec{x}) : \mathbb{R}^d \to \mathbb{R}^1$
- To evaluate $H(\vec{x})$, compute result of layer i, use as inputs for layer $i + 1$.
  - Let $z_j^{(i)}$ be the output of node j in layer i. (layer index; neuron index) = hidden representation
  - Make a vector of these outputs: $\vec{z}^{(i)} = (z_1^{(i)}, z_2^{(i)}, \ldots)^T$
  - ✓ Observe that $\vec{z}^{(i)} = [W^{(i)}]^T \vec{z}^{(i-1)} + \vec{b}^{(i)}$ (input: $\vec{z}^{(0)}$)

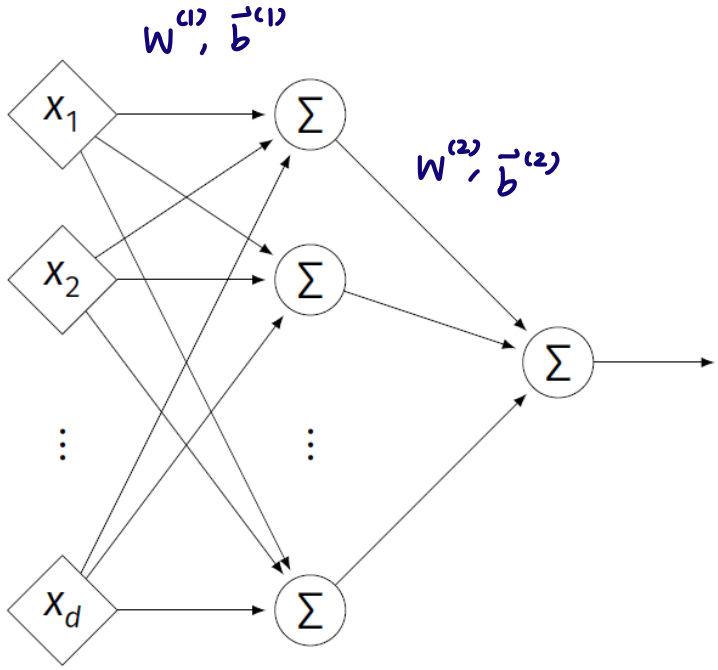
    Current layer i; previous layer $i-1$
- $W_{jk}^{(i)}$ denotes weight of connection between neuron j in layer $(i - 1)$ and neuron k in layer i

output layer: $\vec{z}^{(3)} = W^{(3)T}\vec{z}^{(2)} + \vec{b}^{(3)}$



Neural Networks (more general form)





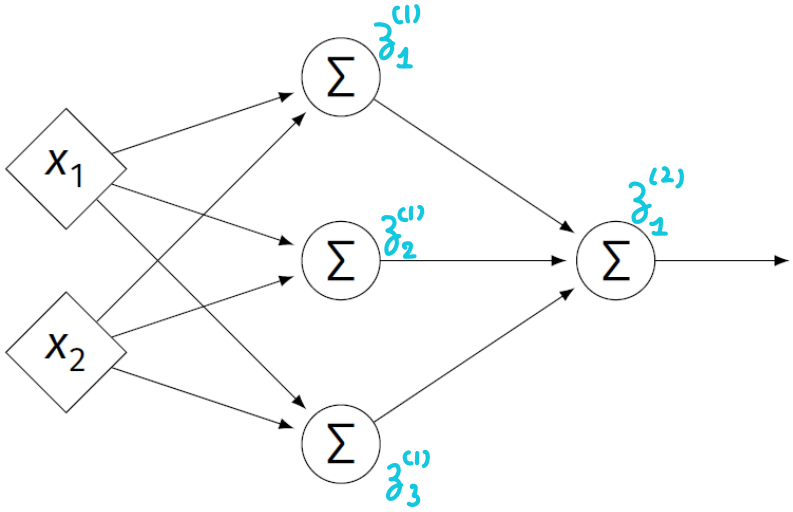
Typically, we'll not draw the bias term & weight, but they're there (described by the network parameters)

$$\Rightarrow \begin{cases} W^{(1)} = [\quad], & W^{(2)} \\ \vec{b}^{(1)} = [\quad], & \vec{b}^{(2)} \end{cases}$$

Example

$$\vec{z}^{(i)} = W^{(i)T}\vec{z}^{(i-1)} + \vec{b}^{(i)} \rightarrow \vec{z}^{(1)} = W^{(1)T}\vec{z}^{(0)} + \vec{b}^{(1)}$$

$$= \begin{bmatrix} 2 & 4 \\ -1 & 5 \\ 0 & 2 \end{bmatrix}\begin{bmatrix} 3 \\ -1 \end{bmatrix} + \begin{bmatrix} 3 \\ -2 \\ -2 \end{bmatrix} = \begin{bmatrix} 2 \\ -8 \\ -2 \end{bmatrix} + \begin{bmatrix} 3 \\ -2 \\ -2 \end{bmatrix} = \begin{bmatrix} 5 \\ -10 \\ -4 \end{bmatrix}$$



$$W^{(1)} = \begin{pmatrix} 2 & -1 & 0 \\ 4 & 5 & 2 \end{pmatrix} \quad W^{(2)} = \begin{pmatrix} 3 \\ 2 \\ -4 \end{pmatrix}$$

$$\vec{b}^{(1)} = (3, -2, -2)^T \quad \vec{b}^{(2)} = (-4)^T$$

Suppose input:

- $\vec{x} = (3, -1)^T = \vec{z}^{(0)}$

$$\vec{z}^{(2)} = W^{(2)T}\vec{z}^{(1)} + \vec{b}^{(2)} = \begin{bmatrix} 3 & 2 & -4 \end{bmatrix}\begin{bmatrix} 5 \\ -10 \\ -4 \end{bmatrix} + -4$$

$$= 11 - 4 = 7$$

Calculate:

- $z_1^{(1)} = 5$
- $z_2^{(1)} = -10$
- $z_3^{(1)} = -4$
- $z_1^{(2)} = 7$

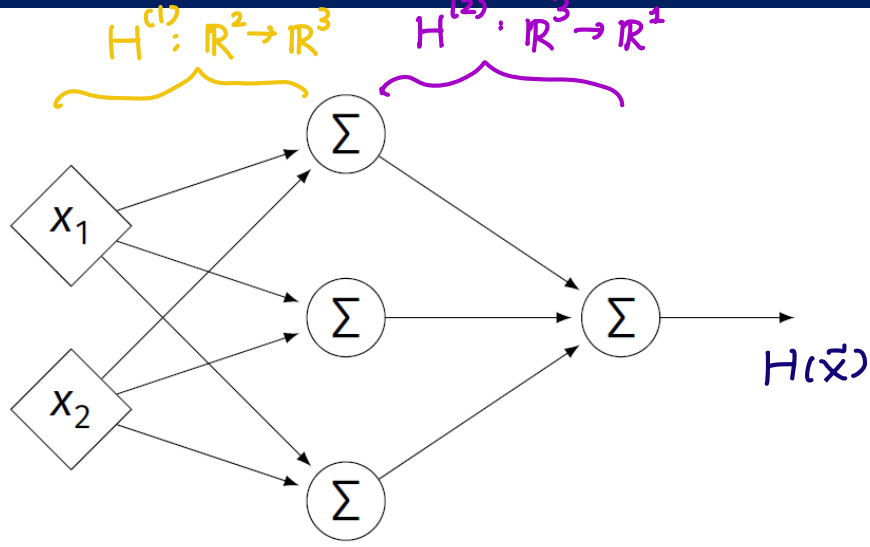
Example

$\vec{z}^{(i)} = W^{(i)T} \vec{z}^{(i-1)} + \vec{b}^{(i)}$

current layer | previous layer

Each Layer is a Function

- We can think of each layer as a function mapping a vector to a vector.
- $H^{(1)}(\vec{z}) = \left[W^{(1)}\right]^T \vec{z} + \vec{b}^{(1)}$
  - $H^{(1)} : \mathbb{R}^2 \to \mathbb{R}^3$
- $H^{(2)}(\vec{z}) = \left[W^{(2)}\right]^T \vec{z} + \vec{b}^{(2)}$
  - $H^{(2)} : \mathbb{R}^3 \to \mathbb{R}^1$



$$H(\vec{x}) = H^{(2)}(H^{(1)}(\vec{x})) = \left[W^{(2)}\right]^T \underbrace{\left(\left[W^{(1)}\right]^T \vec{x} + \vec{b}^{(1)}\right)}_{\vec{z}^{(1)}} + \vec{b}^{(2)}$$

$$= \underbrace{W^{(2)T} W^{(1)T}}_{\tilde{W}^T} \vec{x} + \underbrace{W^{(2)T} \vec{b}^{(1)} + \vec{b}^{(2)}}_{\tilde{b}}$$

Show that:

$$H(\vec{x}) = \left[W^{(2)}\right]^T \left(\left[W^{(1)}\right]^T \vec{x} + \vec{b}^{(1)}\right) + \vec{b}^{(2)} = \vec{w} \cdot \text{Aug}(\vec{x})$$

for some appropriately-defined vector $\vec{w}$.

$$= \tilde{W}^T \vec{x} + \tilde{b}$$

networks so far are linear function!

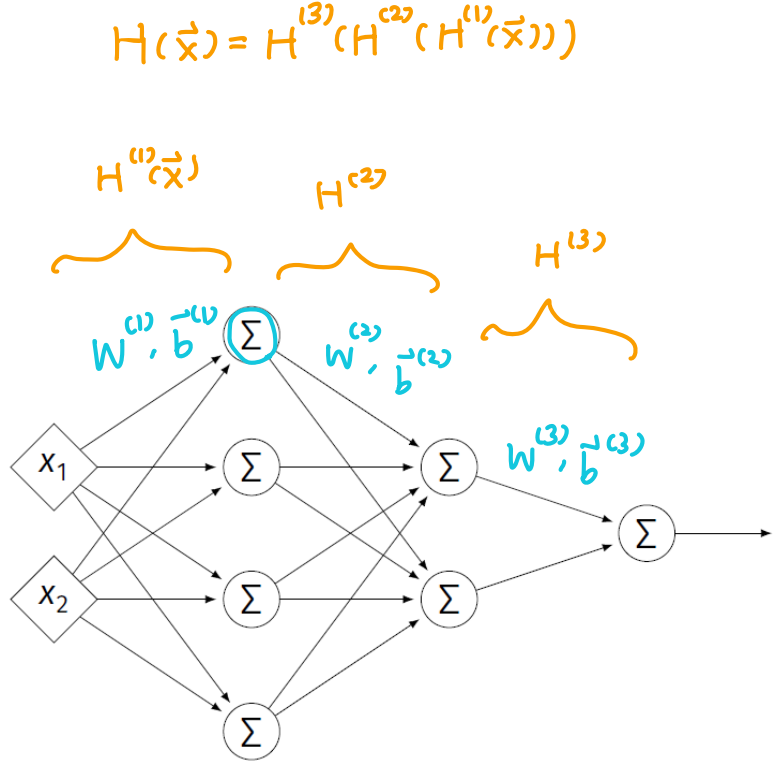
Neural Networks

NNs as Function Composition

- In general, if there k hidden layers:

$$H(\vec{x}) = H^{(k+1)}\left(\cdots H^{(3)}\left(H^{(2)}\left(H^{(1)}(\vec{x})\right)\right)\cdots\right)$$

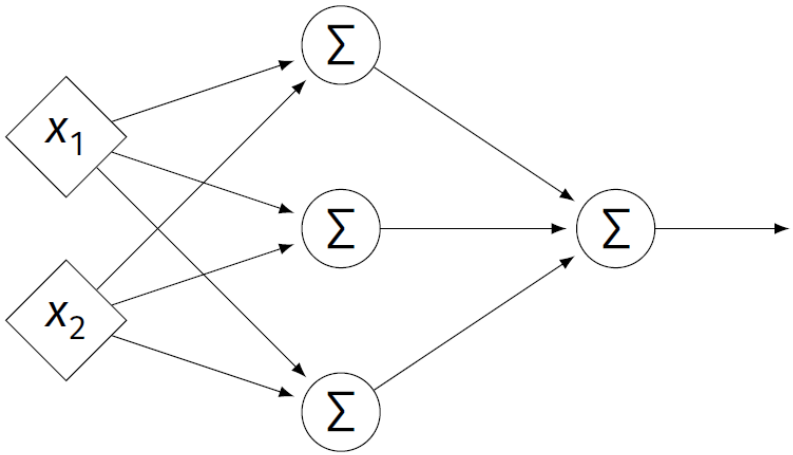
- The composition of linear functions is again a linear function.
- The NNs we have seen so far are all equivalent to linear models!
- For NNs to be more useful, we will need to add **non-linearity**.



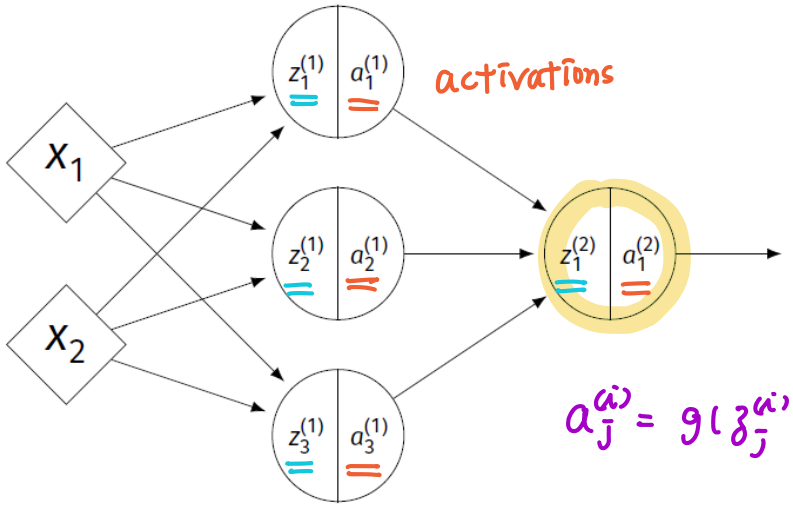
Activation

$a_j^{(i)}$ ← layer index
↖ neuron index

(linear NN)



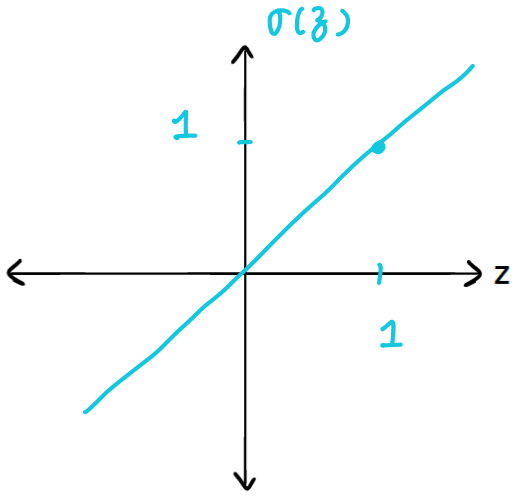
(non-linear NN)



- $z_j^{(i)}$ is the linear activation before g is applied.
- $a_j^{(i)} = g(z_j^{(i)})$ is the actual output of the neuron.

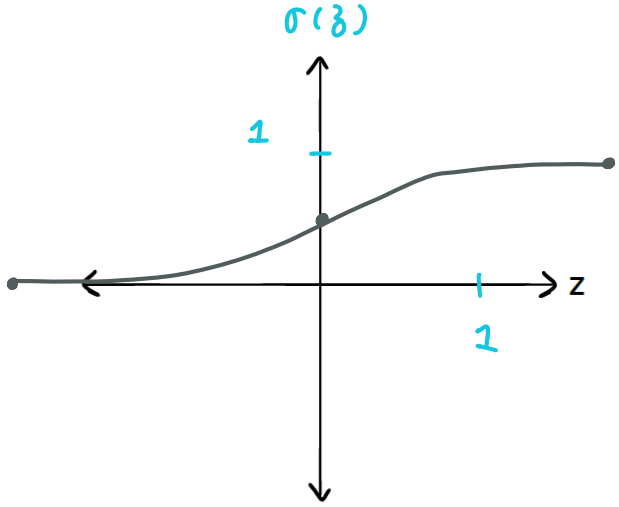
$g(\cdot)$: activation function (sometimes $\sigma(\cdot)$)

Activation



$\sigma(z) = z$

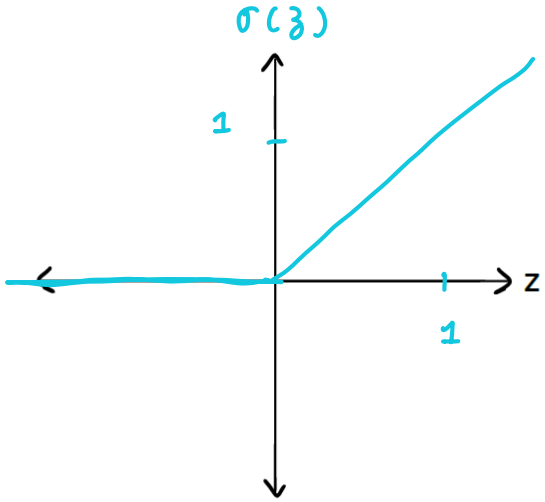
- The **linear** activation is what we've been using.



$\sigma(z) = \frac{1}{1 + e^{-z}}$

- $\sigma(0) = \frac{1}{1+e^{0}} = \frac{1}{2}$
- $\sigma(\infty) = \frac{1}{1+e^{-\infty}} = 1$
- $\sigma(-\infty) = \frac{1}{1+e^{\infty}} = 0$

- The **sigmoid** models saturation in many natural processes.



$g(z) = \max\{0, z\} = \begin{cases} z, & z \geq 0 \\ 0, & z < 0 \end{cases}$

- The **Rectified Linear Unit (ReLU)** tends to work better in practice.

Activation

- The activation of the output neuron(s) can be different than the activation of the hidden neurons.
- In classification, **sigmoid** activation makes sense.
- In regression, **linear** activation makes sense.

(output $H_f = a_1^{(2)} = z_1^{(2)}$)

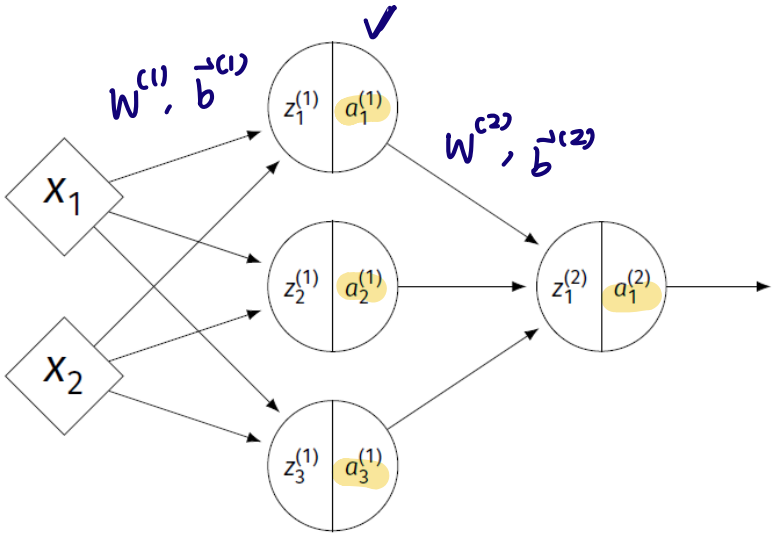
Main Idea

A neural network with linear activations is a linear model. If non-linear activations are used, the model is made non-linear.

Example

$$\vec{z}^{(1)} = W^{(1)T}\vec{a}^{(0)} + \vec{b}^{(1)}$$

$$= \begin{bmatrix} 2 & 4 \\ -1 & 5 \\ 0 & 2 \end{bmatrix}\begin{bmatrix} 3 \\ -1 \end{bmatrix} + \begin{bmatrix} 3 \\ -2 \\ -2 \end{bmatrix} = \begin{bmatrix} 2 \\ -8 \\ -2 \end{bmatrix} + \begin{bmatrix} 3 \\ -2 \\ -2 \end{bmatrix} = \begin{bmatrix} 5 \\ -10 \\ -4 \end{bmatrix}$$



$$W^{(1)} = \begin{pmatrix} 2 & -1 & 0 \\ 4 & 5 & 2 \end{pmatrix} \quad W^{(2)} = \begin{pmatrix} 3 \\ 2 \\ -4 \end{pmatrix}$$

$$\vec{b}^{(1)} = (3, -2, -2)^T \quad \vec{b}^{(2)} = (-4)^T$$

$$\vec{a}^{(1)} = g(\vec{z}^{(1)}) = \begin{bmatrix} 5 \\ 0 \\ 0 \end{bmatrix}$$

- g = ReLU
- Linear output
- $\vec{x} = (3, -1)^T$

$$\vec{z}^{(2)} = W^{(2)T}\underbrace{\vec{a}^{(1)}}_{\vec{z}^{(1)}} + \vec{b}^{(2)} = \begin{bmatrix} 3 & 2 & -4 \end{bmatrix}\begin{bmatrix} 5 \\ 0 \\ 0 \end{bmatrix} + -4$$

$$= 15 - 4 = 11$$

- $z_1^{(1)} = 5$
- $a_1^{(1)} = 5$
- $z_2^{(1)} = -10$
- $a_2^{(1)} = 0$
- $z_3^{(1)} = -4$
- $a_3^{(1)} = 0$
- $z_1^{(2)} = 11$

Feature map

- We have seen how to fit non-linear patterns with linear models via **basis functions** (i.e., a feature map).

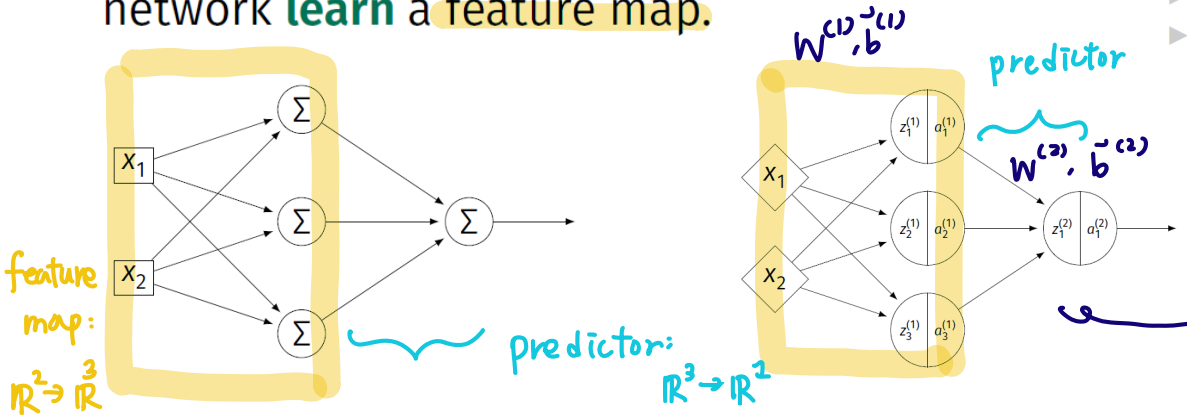
new features

$$H(\vec{x}) = w_0 + w_1\phi_1(\vec{x}) + \ldots + w_k\phi_k(\vec{x})$$

- These basis functions are fixed **before** learning.
- **Downside:** we have to choose $\vec{\phi}$ somehow.

Learning a Feature map

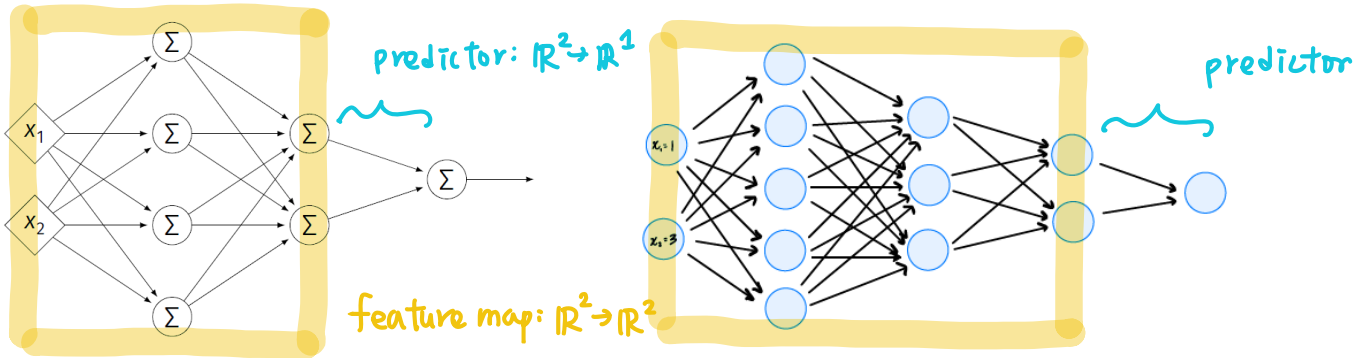
- **Interpretation:** The hidden layers of a neural network **learn** a feature map.



- The hidden layer performs a feature map from $\mathbb{R}^2$ to $\mathbb{R}^3$.
- The output layer makes a prediction in $\mathbb{R}^3$.
- **Intuition:** The feature map is learned so as to make the output layer's job "easier".

$W^{(1)}, \vec{b}^{(1)}$: parameters for features

$W^{(2)}, \vec{b}^{(2)}$: parameters for predictors

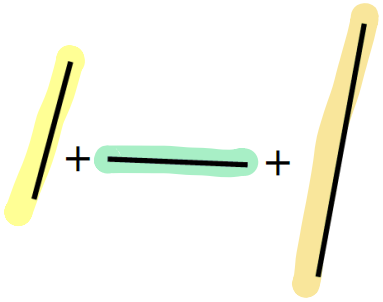


Convolution Neural Network

Image recognition

handwritten digit (Mnist)

- Complex shapes are made of simple patterns
- The human visual system uses this fact

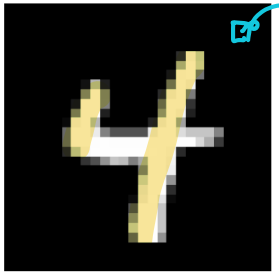


- Line detector → shape detector → ... → face detector
- Can we replicate this with a deep NN?

"detector" (features/representations)

Edge detector

dot product between image patch & filter:
$0\times0+0\times(-1)+255\times1+0\times0+0\times(-1)+1\times(255)$
$+0\times0+0\times(-1)+255\times1 = 255\times3$, large #



pixel value
0 ~ 255

- How do we find **vertical edges** in an image?
- One solution: **convolution** with an **edge filter**.

detect edges in the image
vertical / horizontal / diagonal

(3×3)

image patch:

0	0	255
0	0	255
0	0	255

*

filter:

0	-1	1
0	-1	1
0	-1	1

(detect vertical line @ right boundary)

detect transition (from no edge to edge)

image patch:

0	0	0
0	0	0
0	0	0

*

filter:

0	-1	1
0	-1	1
0	-1	1

dot product: 0 (small #)

image patch:

255	0	0
255	0	0
255	0	0

*

filter:

0	-1	1
0	-1	1
0	-1	1

dot product: 0 (small #)

Convolutions

- Move the filter over the entire image, repeat procedure.

full image : 8 x 8

INPUT (2D input)

0	0	0	0	0	0		
0	0	.9	0	0	.7		
0	0	.9	0	0	.8		
0	0	.8	0	0	.9		
0	0	.7	0	0	0		

*

0	-1	1
0	-1	1
0	-1	1

edge filter (3x3)

=

OUTPUT (Response map / activation map)

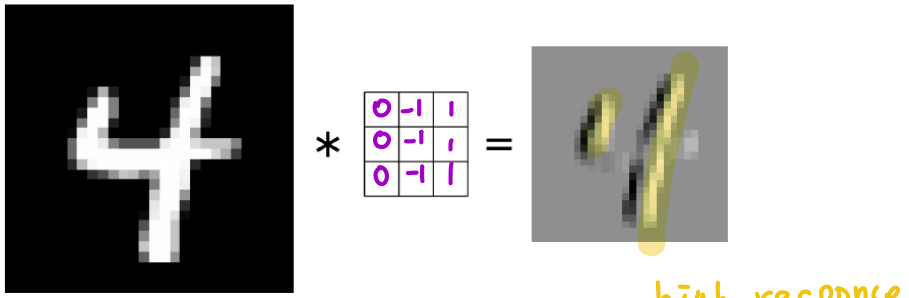
1.8	-1.8	0	1.5	v	v
2.6	-2.6	v	v	v	v
v	v	v	v	v	v
v	v	v	v	v	v

2D map

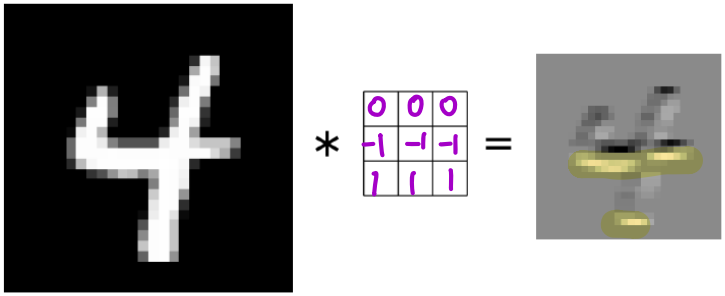
Convolutions

- The result is the (2d) **convolution** of the filter with the image.
- Output is also 2-dimensional array.
- Called a **response map**.

Vertical Filter



Horizontal Filter

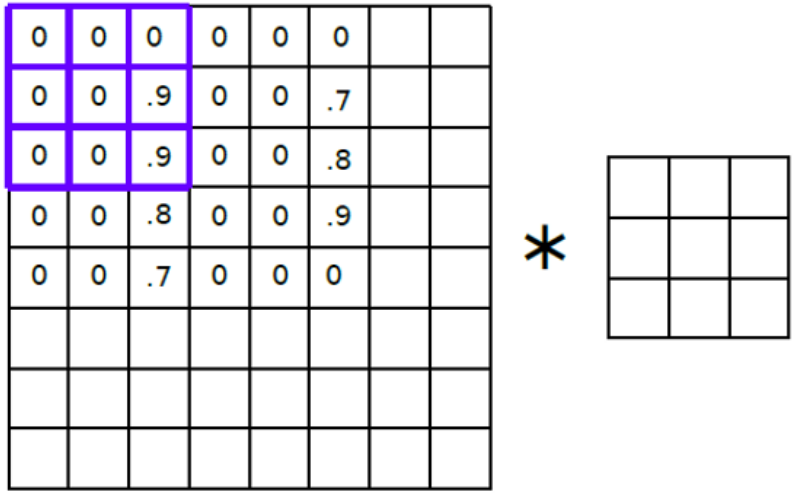


Filters

- Typically 3×3 or 5×5.
- Variations: different **stride**, image **padding**.

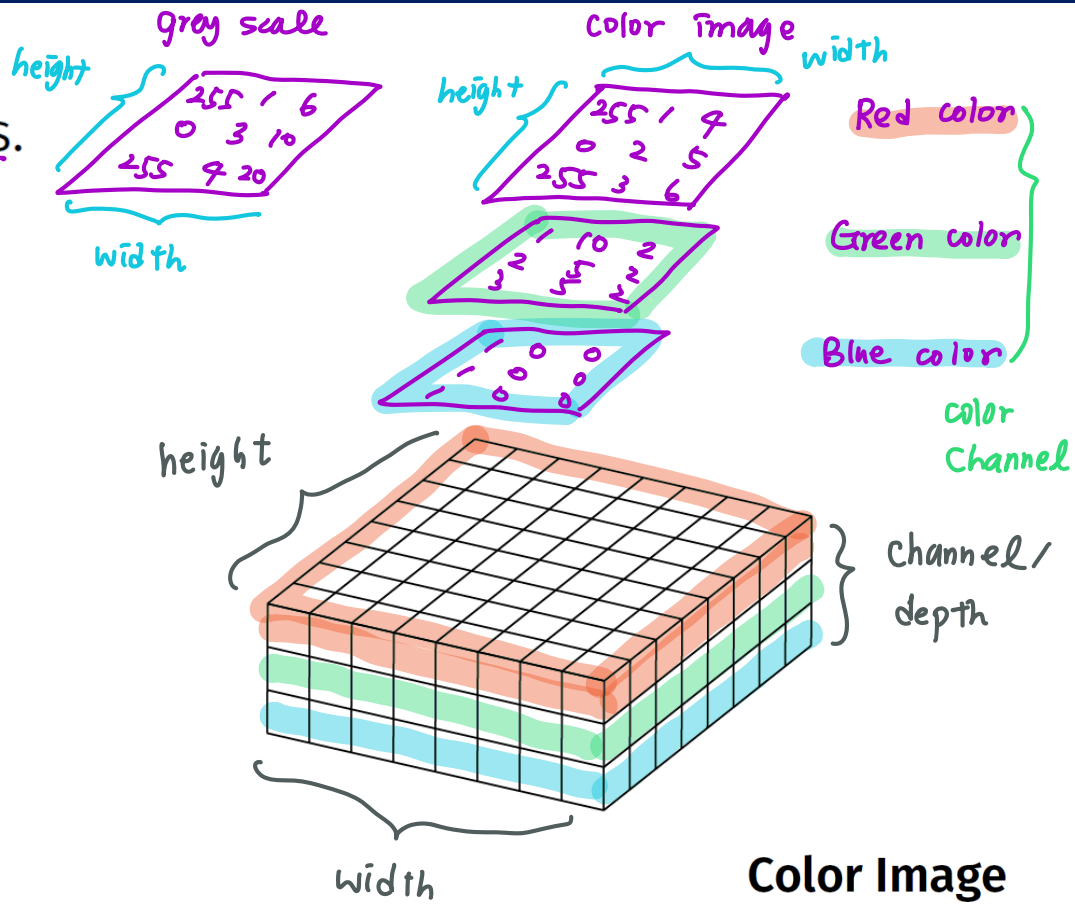
another hyper parameter in convolution

(stride = 1 : move 1 pixel → hyper parameter control how far to move)



Filters

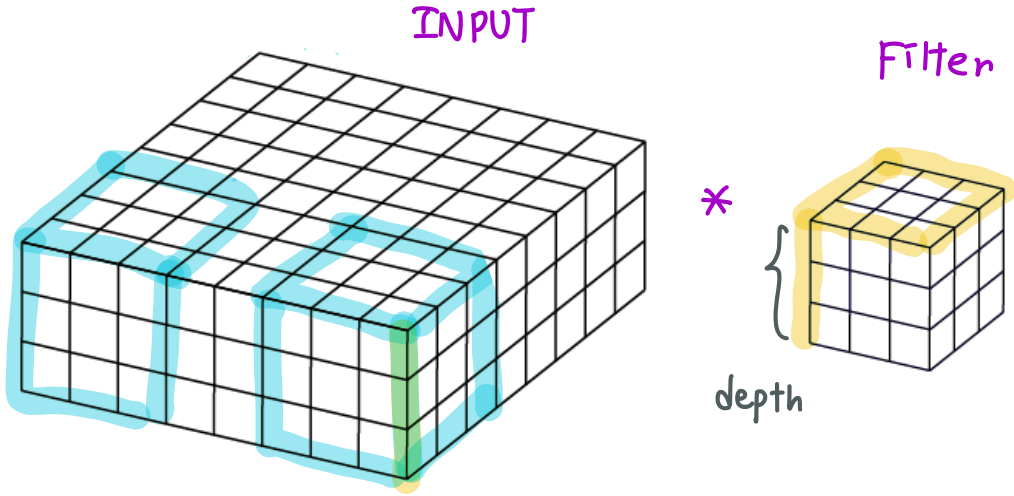
- Black and white images are 2-d arrays.
- But color images are 3-d arrays:
  - a.k.a., **tensors**
  - Three color **channels**: red, green, blue.
  - height × width × 3
- How does convolution work here?



Filters

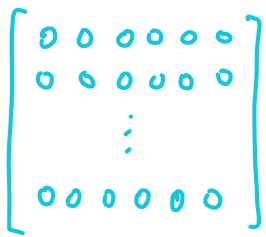
- The filter must also have three channels:
  - $3 \times 3 \times 3$, $5 \times 5 \times 3$, etc.

depth of filter



- Filter must have same number of channels as image.
  - 3 channels if image RGB.
- Result is still a 2-d array.

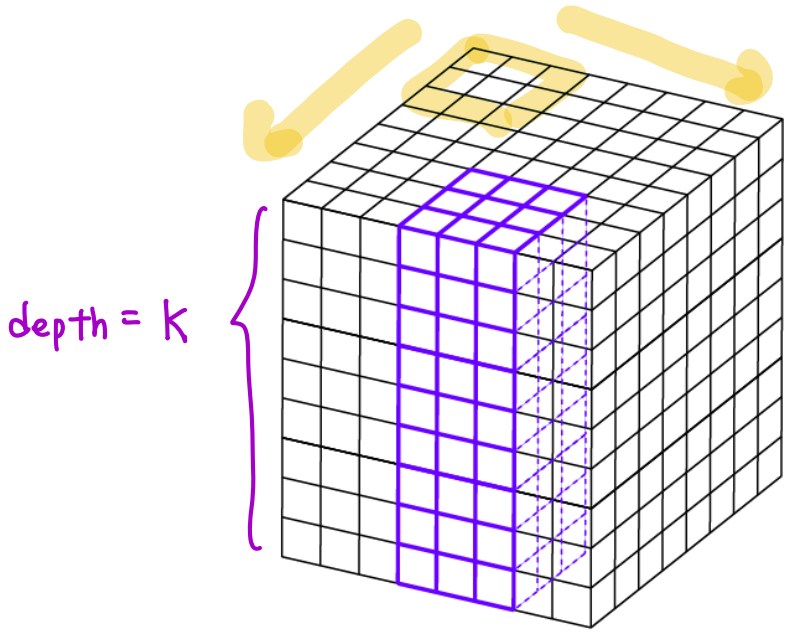
OUTPUT (2D array)



Filters

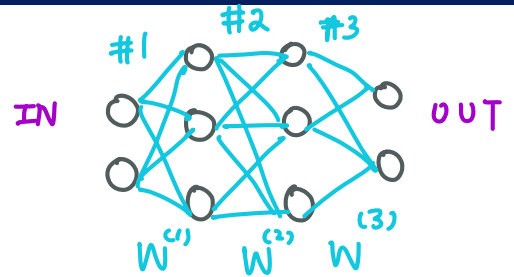
General Case

- Input "image" has k channels.
- Filter must have k channels as well.
  - e.g., $3 \times 3 \times k$
- Output is still 2 – d



Convolutional Neural Networks (CNNs)

- **CNNs** are the state-of-the-art for many computer vision tasks
- **Idea**: use convolution in early layers to create new feature representation.
- **But!** Filters are **learned**.

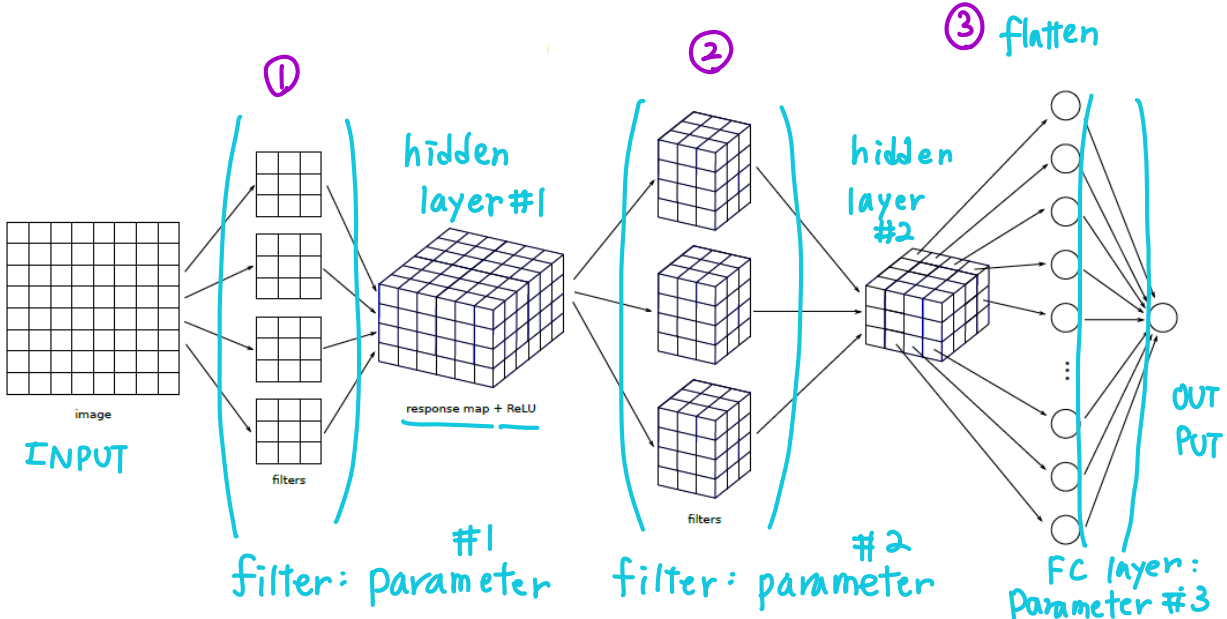


earlier: define by hand

0	-1	1
0	-1	1
0	-1	1

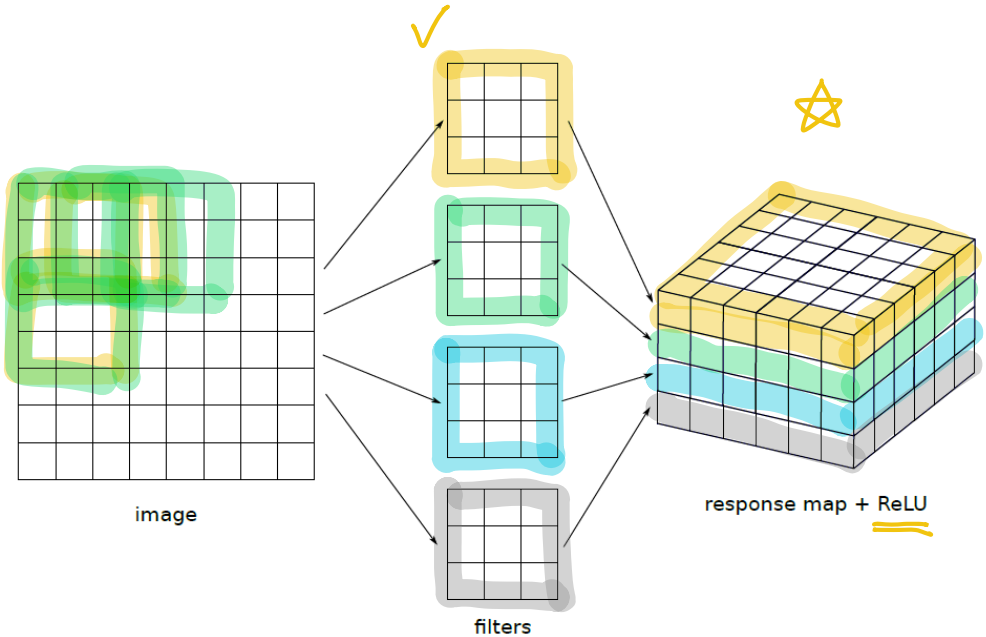
Now: Learned

?	?	?
?	?	?
?	?	?



Convolutional Neural Networks (CNNs)

① Input Convolutional Layer

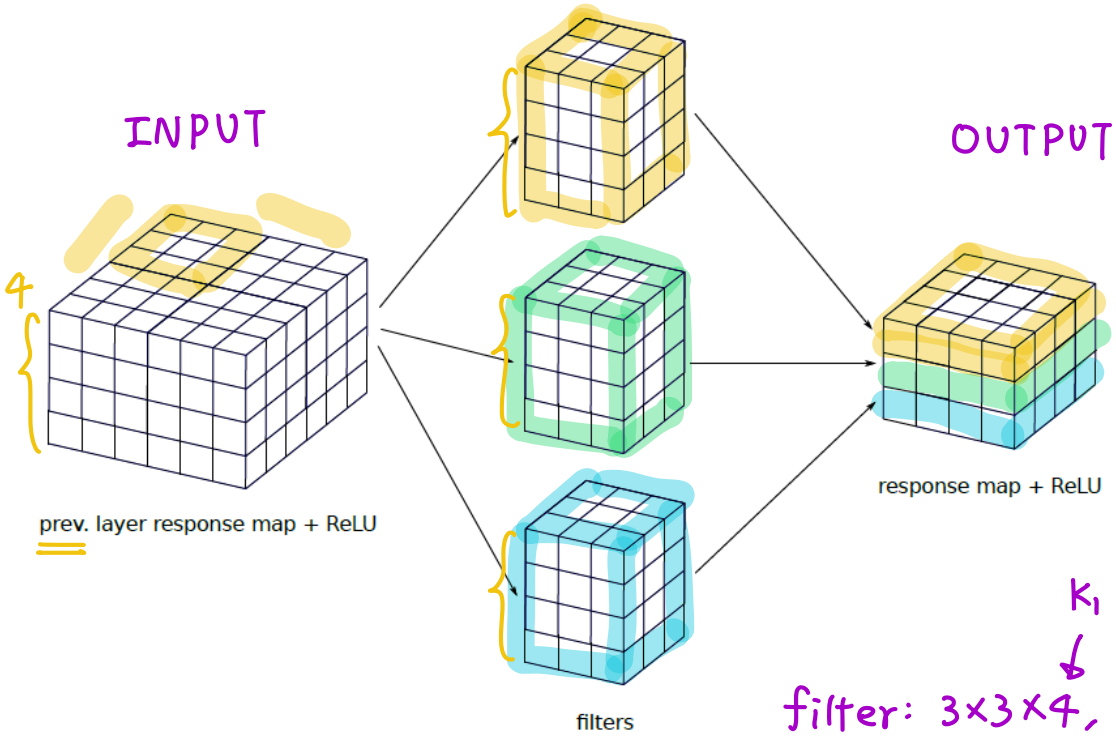


- Input image with one channel (grayscale)
- k_1 filters of size $\ell \times \ell \times 1$
- Results in k_1 convolutions, stacked to make response map.
- ReLU (or other nonlinearity) applied entrywise.

filter: 3x3x1, 4 filters ($k_1 = 4$, $\ell = 3$)

Convolutional Neural Networks (CNNs)

② Second Convolutional Layer



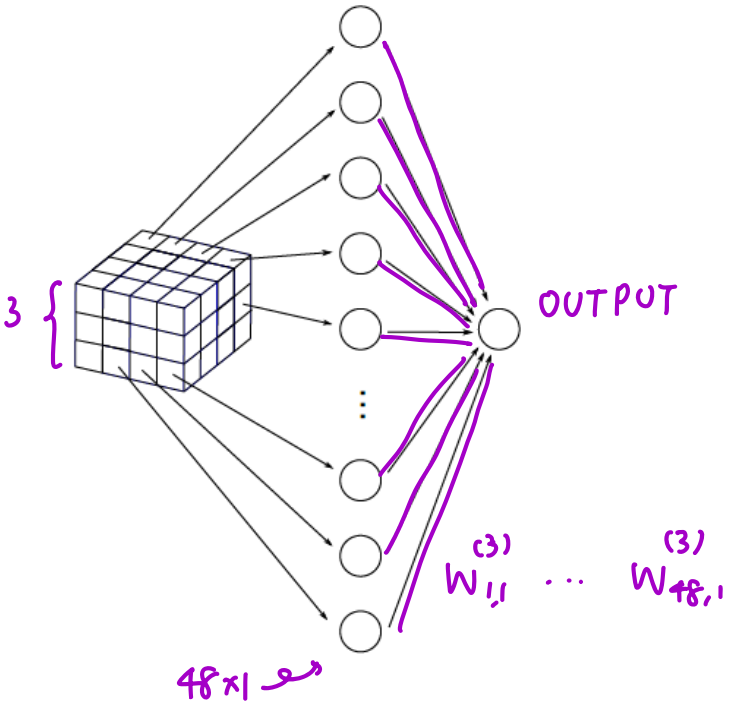
- Input is a 3-d **tensor**.
  - "Stack" of k_1 response maps.
- k_2 filters, each a 3-d tensor with k_1 channels.
- Output is a 3-d tensor with k_2 channels.

k_1

filter: $3 \times 3 \times 4$, $k_2 = 3$,

Convolutional Neural Networks (CNNs)

③ **Flattening**: stretch the tensor into 1-D vector

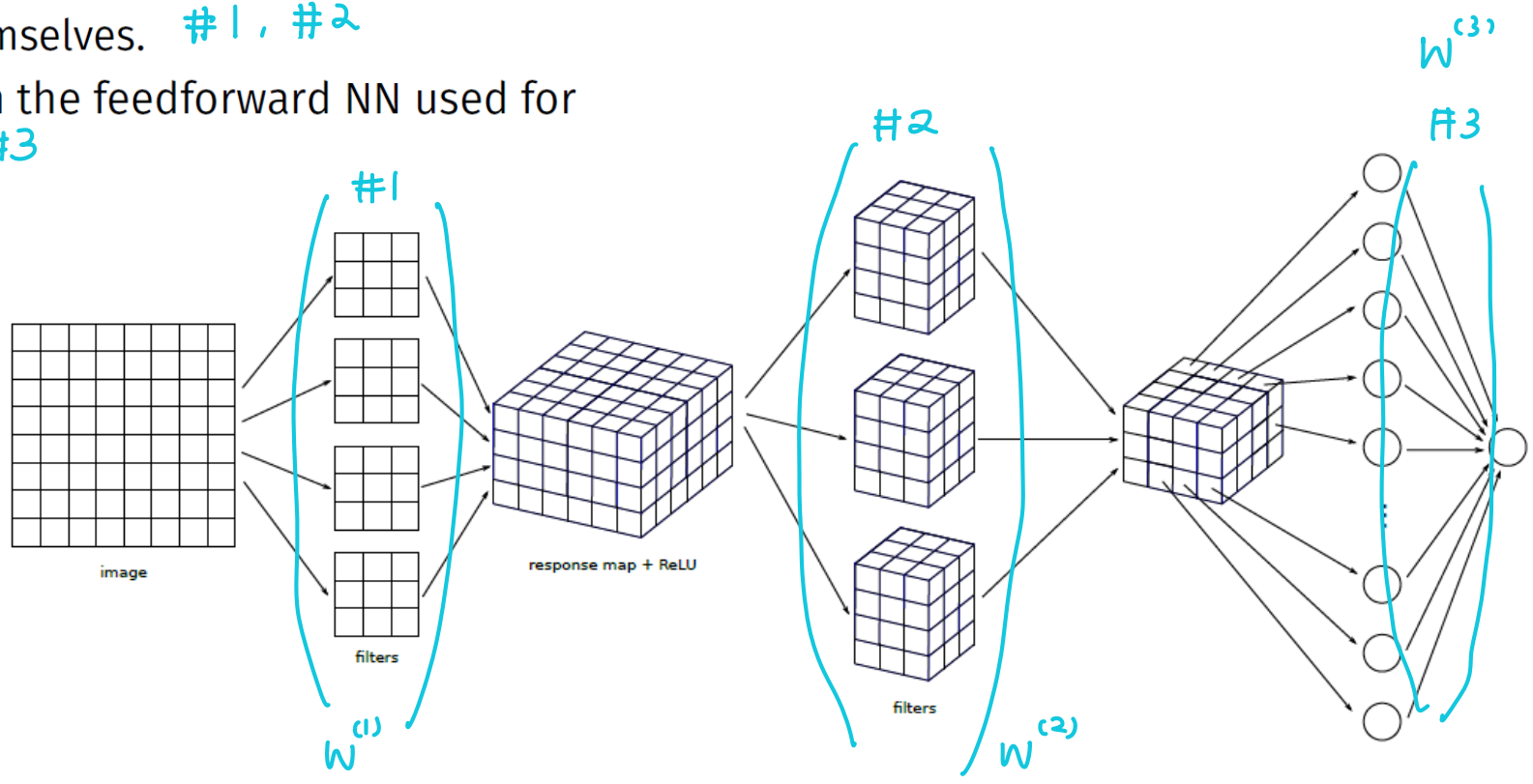


- May add more convolutional layers.
- Last convolutional layer used as input to a feedforward, fully-connected network.
- Need to "flatten" the output tensor.

Convolutional Neural Networks (CNNs)

What is learned?

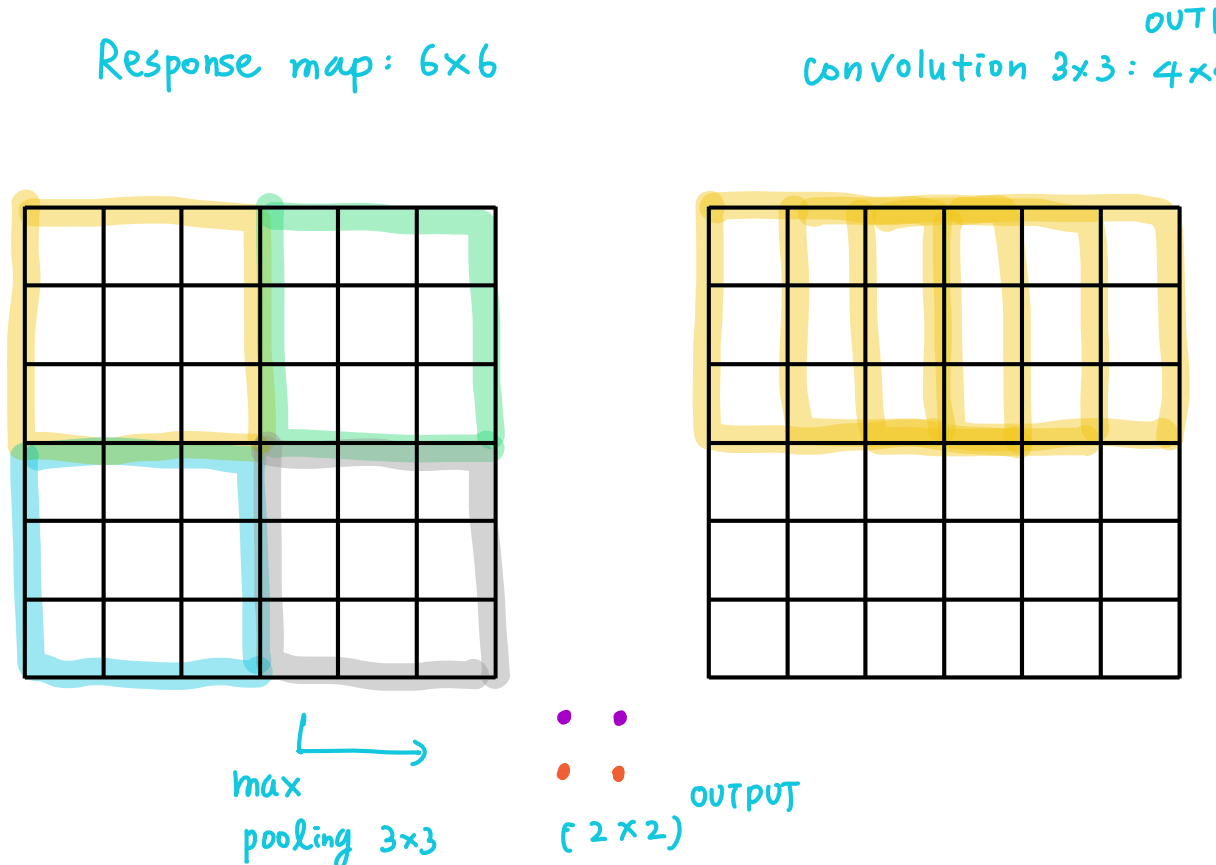
- The filters themselves. #1, #2
- The weights in the feedforward NN used for prediction. #3



Convolutional Neural Networks (CNNs)

Max Pooling

- **Max pooling** is an important part of convolutional layers in practice.
- Reduces size of response map, number of parameters.



Example

- Predict whether image is of a **car** or a **truck**.

truck

truck

car

car

Approach #1: Least squares classifier (learn a linear layer)

- Result: 72% train accuracy, 63% test accuracy
- Need a better feature representation

Approach #2: CNN

- 94% train accuracy, 90% test accuracy

Example

Structure of CNN:

- 3 convolutional layers with 32, 64, 64 filters
- ReLU, max pooling after first two
- Dense layer with 64 hidden neurons, ReLU
- Output layer with sigmoid activation
- Minimize cross-entropy loss; use *dropout*

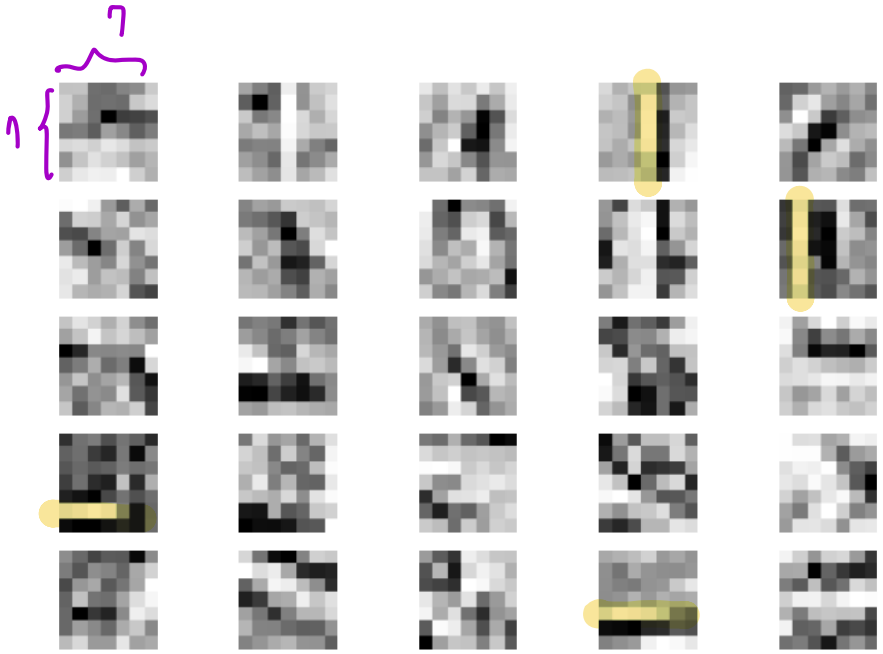
define model structure of CNN

```
model = keras.models.Sequential()

model.add( keras.layers.Conv2D(32, (7, 7), activation='relu', input_shape=(32, 32, 1)))
model.add(keras.layers.MaxPooling2D((2, 2)))

model.add(keras.layers.Conv2D(64, (5, 5), activation='relu'))
model.add(keras.layers.MaxPooling2D((2, 2)))

model.add(keras.layers.Conv2D(64, (3, 3), activation='relu'))

model.add(keras.layers.Flatten())
model.add(keras.layers.Dropout(0.5))
model.add(keras.layers.Dense(64, activation='relu'))
model.add(keras.layers.Dense(1, activation='sigmoid'))


model.compile(
    optimizer=keras.optimizers.RMSprop(),
    loss=keras.losses.BinaryCrossentropy(),
    metrics=['accuracy']
)

model.fit(
    X_train,
    y_train,
    epochs=30,
    validation_data=(X_test, y_test)
)
```

train the model

Example

Filters (learned from training CNNs!)




```
model = keras.models.Sequential()

model.add( keras.layers.Conv2D(32, (7, 7), activation='relu', input_shape=(32, 32, 1)))
model.add(keras.layers.MaxPooling2D((2, 2)))

model.add(keras.layers.Conv2D(64, (5, 5), activation='relu'))
model.add(keras.layers.MaxPooling2D((2, 2)))

model.add(keras.layers.Conv2D(64, (3, 3), activation='relu'))

model.add(keras.layers.Flatten())
model.add(keras.layers.Dropout(0.5))
model.add(keras.layers.Dense(64, activation='relu'))
model.add(keras.layers.Dense(1, activation='sigmoid'))
```

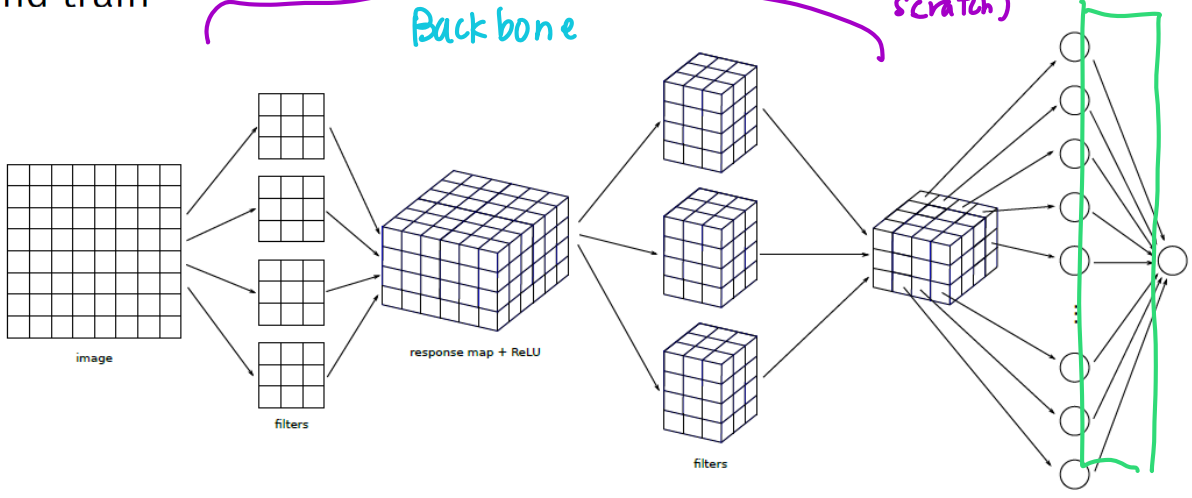
Next steps

- In practice, you might not train your own CNN (from scratch)
- Instead, take "pre-trained" convolutional layers from a much bigger network
- Attach untrained fully-connected layer and train
- This is **transfer learning**
  can (finetune feature / back bone)

can take from pre-trained network (no need to learn/ build from scratch)

Back bone

learn final weight



Autoencoder

Representation learning

- At a high level, representation learning finds an **encoding function** $\text{encode}(\vec{x}) : \mathbb{R}^d \to \mathbb{R}^k$.
  - Ideally, this function captures useful aspects of the data distribution.
  - Encoding can decrease dimensionality.
  - Intuitively, we may want to preserve as much "information" about $\vec{x}$ as possible.
- We should be able to **decode** the encoding and **reconstruct** the original point, approximately.

$$\vec{x} \approx \text{decode}(\text{encode}(\vec{x}))$$

  - In general, $\text{decode}(\text{encode}(\vec{x}))$ will not be exactly equal to $\vec{x}$.

$\vec{x} \in \mathbb{R}^d$ (d-dimensional data)

$\vec{z} \in \mathbb{R}^k$ (k-dimensional feature (new representation))

$k < d$

"reconstruction"

n data points

reconstruction error:

$$\sum_{i=1}^{n} \|\vec{x}^{(i)} - \text{decode}(\text{encode}(\vec{x}^{(i)}))\|^2$$

Reconstruction

Representation learning

- Of course, it is trivial to find an encoder/decoder with zero reconstruction error:

$$\text{encode}(\vec{x}) = \vec{x} = \text{decode}(\vec{x})$$

reconstruction error:

$$\sum_{i=1}^{n} \|\vec{x}^{(i)} - \text{decode}(\text{encode}(\vec{x}^{(i)}))\|^2$$

  - Such an encoder is not useful.
- To avoid trivial solution:
- Instead, we constrain the form of the encoder so that it cannot simply copy the input.

Example: PCA

- In PCA, we **encode** a point $\vec{x}$ by projecting it onto the top k eigenvectors of data covariance matrix:

$$\text{encode}(\vec{x}) = U^T\vec{x}$$

$$\text{decode}(\vec{z}) = U\vec{z}$$

- **Goal:** find U to minimize reconstruction error on a dataset $\vec{x}^{(1)}, \ldots, \vec{x}^{(n)}$.
- **Solution:** pick columns of U to be top k eigenvectors of data covariance matrix.

Representation learning

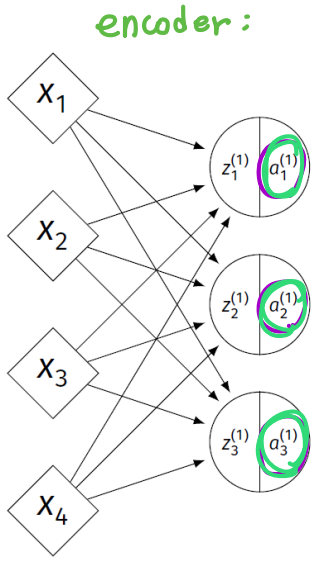
Example: PCA

- encode($\vec{x}$) = $U^T\vec{x}$ is a linear encoding function.
- What if we let encode be nonlinear? That is, let's generalize PCA.

Encoder as a Neural Network

encoder($\vec{x}$) = $\vec{z} \in \mathbb{R}^k$

- Assume encode($\vec{x}$) is a (deep) **neural network**.
  - Can use nonlinear activations, have more than one layer.
- Output is not a single number, but k numbers.
  - I.e., a vector in $\mathbb{R}^k$
  - The output of the encoder is the new representation. $\vec{z}$
- To train the encoder, we'll need a **decoder**.



Representation learning

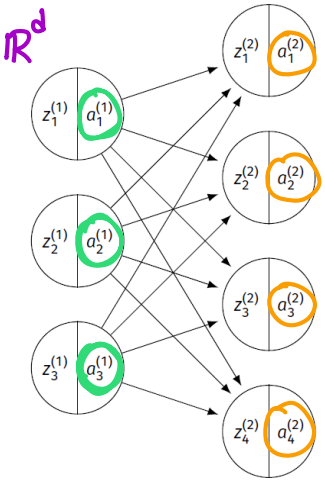
Decoder as a Neural Network

decoder($\vec{z}$) = $\hat{\vec{x}} \in \mathbb{R}^d$

$\vec{z} \in \mathbb{R}^k$

- Assume decode($\vec{z}$) is a (deep) **neural network**.
  - Can use nonlinear activations, have more than one layer.
- Output is not a single number, but d numbers.
  - Same dimensionality as original input, $\vec{x}$.
  - I.e., a vector in $\mathbb{R}^d$

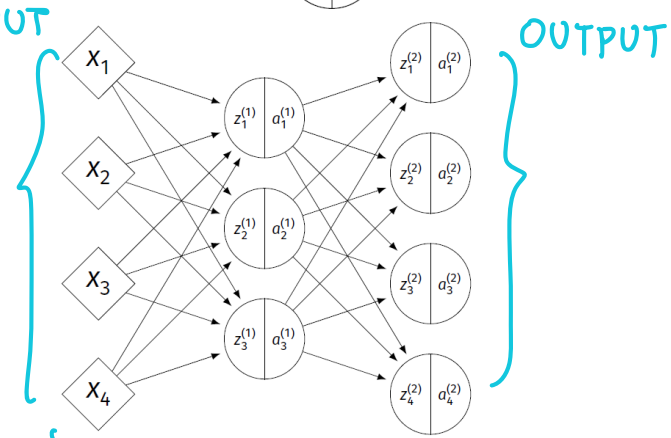
decoder:



d=4, k=3

decode(encode($\vec{x}$)) as a NN

- Together, decode(encode($\vec{x}$)) is a neural network $H(\vec{x}) : \mathbb{R}^d \to \mathbb{R}^d$.



Training

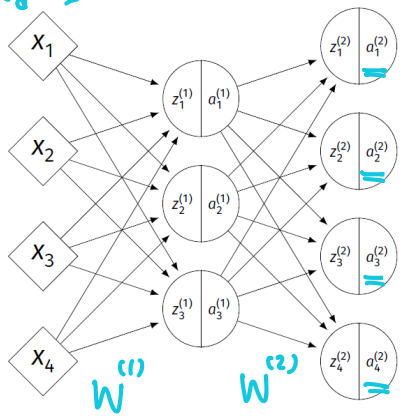
- We want $H(\vec{x}) \approx \vec{x}$
- One approach: train network to minimize reconstruction error.

$$\sum_{i=1}^{n} \|\vec{x}^{(i)} - H(\vec{x}^{(i)})\|^2 = \sum_{i=1}^{n} \sum_{j=1}^{d} (\vec{x}_j^{(i)} - (H(\vec{x}^{(i)}))_j)^2$$

$$= \sum_{i=1}^{n} \sum_{j=1}^{d} (\vec{x}_j^{(i)} - a_j^{(2)}(\vec{x}^{(i)}))^2$$

$H(\vec{x})$

$$\vec{x}^{(i)} = \begin{bmatrix} x_1^{(i)} \\ x_2^{(i)} \\ \vdots \\ x_d^{(i)} \end{bmatrix}$$



reconstruction error:

$$\sum_{i=1}^{n} \|\vec{x}^{(i)} - \text{decode}(\text{encode}(\vec{x}^{(i)}))\|^2$$

- The network can be trained using gradient-based methods.
  - E.g., stochastic gradient descent.
- **Note:** this is an **unsupervised** learning problem. 'i' we didn't use "y" (label)

Autoencoders

- When the encoder/decoder are NNs, $H(\vec{x})$ = decode(encode($\vec{x}$)) is an **autoencoder**.

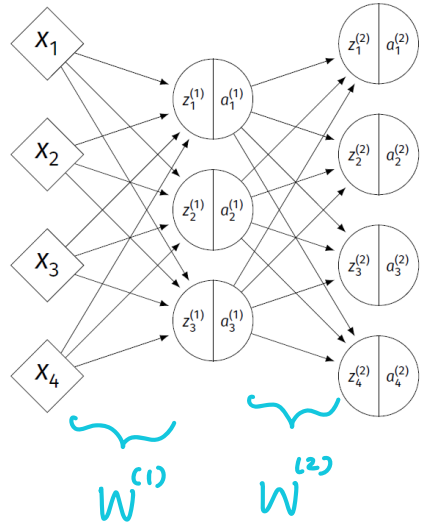
- We can view autoencoders as generalizations of PCA.

  encoder that performs an orthogonal projection:

  $$\text{encode}(\vec{x}) = U^T\vec{x}$$

  $$\text{decode}(\vec{z}) = U\vec{z}$$

  - encode/decode are neural networks (with linear activations). (i.e. $a^{(i)} = z^{(i)}$)



Exercise

Exercise

True/False: training an autoencoder to minimize reconstruction error will result in the same encode($\vec{x}$) function as PCA.

Exercise

Answer: False

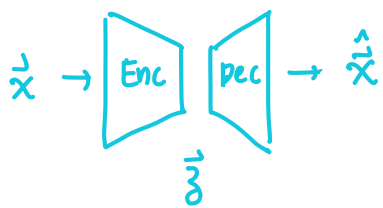
- PCA minimizes reconstruction error **subject to** the constraint that the columns of U are orthonormal.
- Without the orthonormality constraint, the autoencoder learns a different encoding.
- *However*, the autoencoder learns a (non-orthogonal) projection into the same space as PCA.

In other words...

- PCA is an autoencoder trained with an additional orthonormality constraint.
- Cannot easily be learned by gradient descent; find eigenvectors instead.

Use of Autoencoders

- Like PCA, autoencoders can be used for **dimensionality reduction**.
- Unlike PCA, autoencoders can learn nonlinear maps.
- Encoded data can be used as input to predictive model, etc.

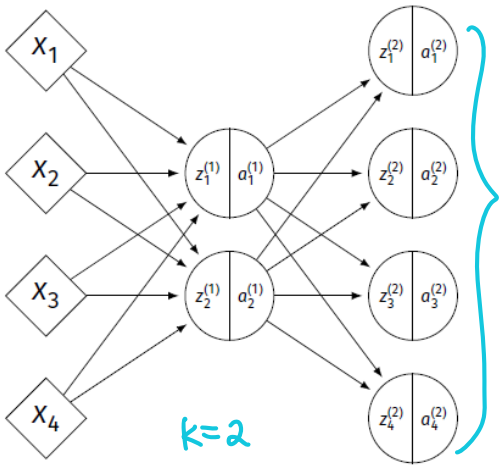


Use of Autoencoders

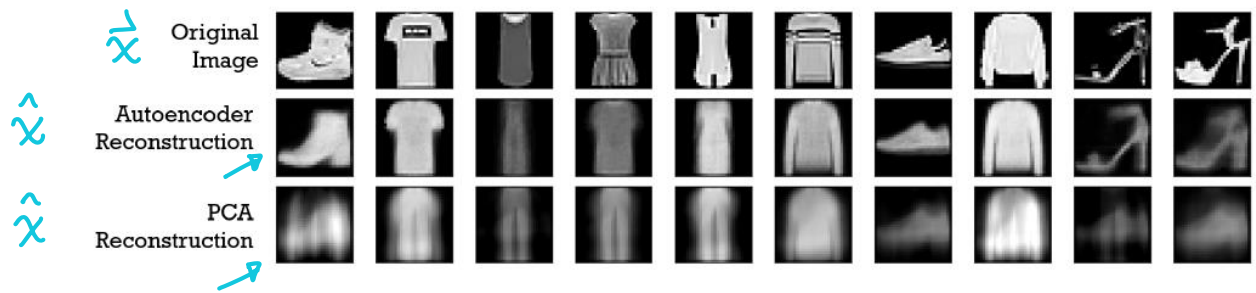
Dimensionality Reduction $(K < d)$

undercomplete autoencoder

number of hidden nodes < d



$d = 4$,



Michela Massi - Own work, CC BY-SA 4.0
https://commons.wikimedia.org/w/index.php?curid=80176900

Use of Autoencoders

Denoising Autoencoders

- **Idea:** train an autoencoder to remove noise.
- Add random noise to each $\vec{x}^{(i)}$ to get $\tilde{x}^{(i)}$.
- Train network so that $H(\tilde{x}^{(i)}) \approx \vec{x}$.

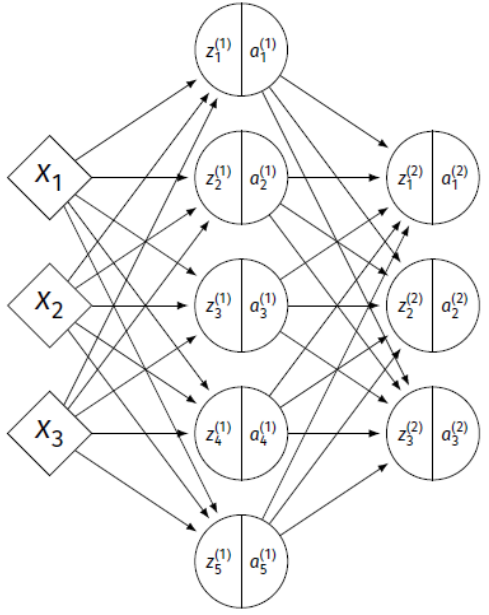
(noisy input)

noisy input

clean input

$K > d$

greater dimensionality



Causality

Association vs. Causality

- Association does not imply causality
- Association: X and Y are associated iff

$$\exists x_1 \neq x_2 \; P(Y|X=x_1) \neq P(Y|X=x_2)$$

Causality: X is a cause of Y iff

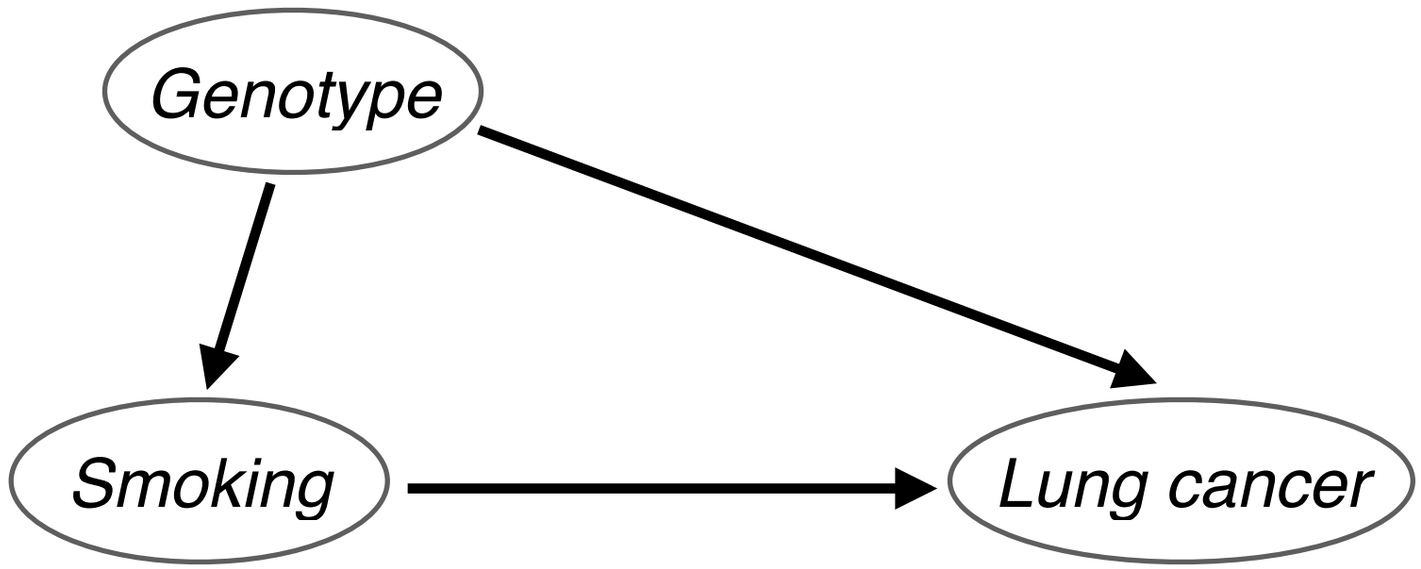
$$\exists x_1 \neq x_2 \; P(Y|do\ (X=x_1)) \neq P(Y|do\ (X=x_2))$$

Do operator: do(X=x_1) means X is manipulated/intervened to x_1

Examples of manipulations: medical interventions (treatments); public policies (e.g., mask mandate); knockout a gene;

Graphical Causal Representations

- A directed graph represents a causally sufficient causal structure

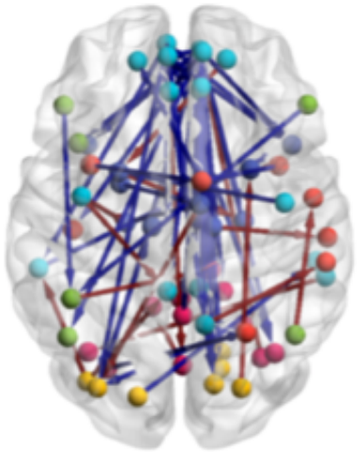


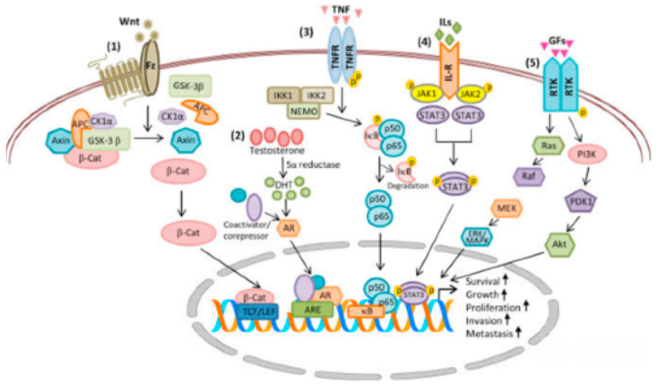
(from "Causation, Prediction, and Search" by SGS)

- Nodes represent random variables
- Directed edge from A to B means A is a direct cause of B relative to the given variable set $\boldsymbol{V}$

Why is Causality Important?

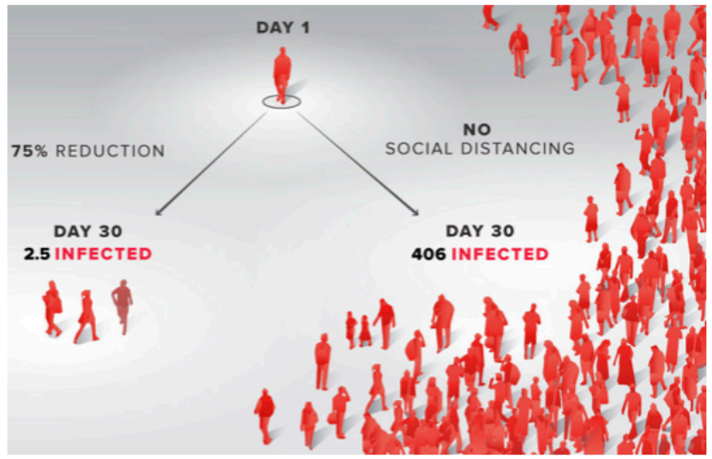
- For understanding mechanisms and processes





Huang, et al., 2019

- For policy making



- Causality-facilitated AI

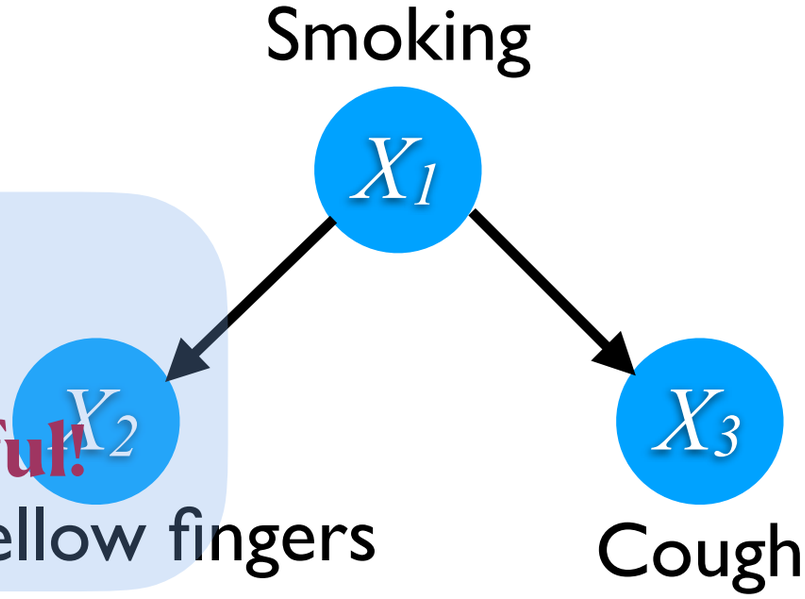
- For control
- For fault detection
- For counterfactuals
- For marketing
- For medical research
- For credit assignment
- ...

Causality for Artificial "Intelligence"

- Three questions in current AI:
  - **Association**: Would the person cough if we *find* he/she has yellow fingers?

    $P(X3 \mid X2=1)$ *Seeing*

    Causality is useful!



  - **Intervention**: Would the person cough if we *make sure* that he/she has yellow fingers?

    $P(X3 \mid \text{do}\ (X2=1))$ *Doing/Intervening*

    Causality is required!

  - **Counterfactual**: Would George cough *had* he had yellow fingers, *given that he does not have yellow fingers and coughs*?

    $P(X3_{X2=1} \mid X2 = 0, X3 = 1)$ *Imagining/Retrospection/ Creativity*

How to Find Causality?

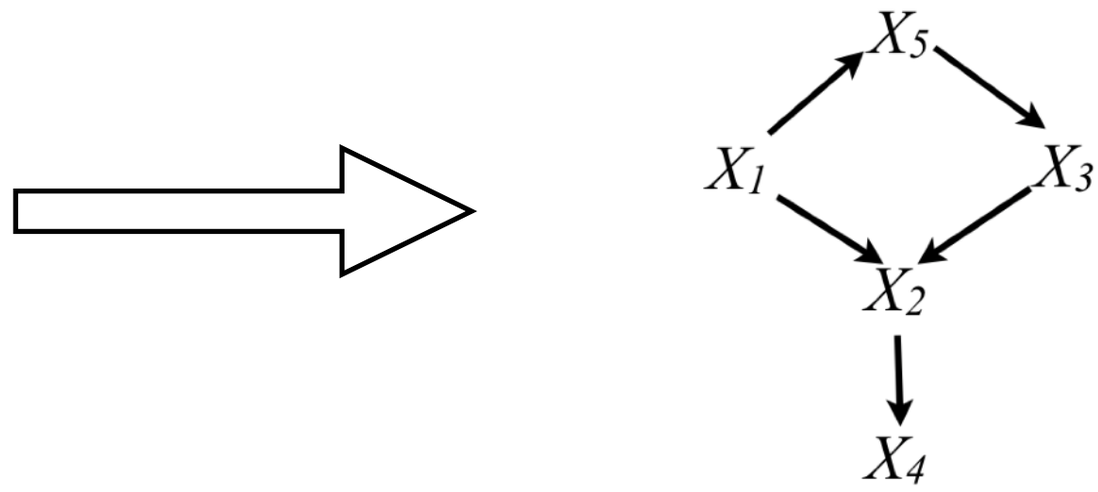
- Association =/⇒ causality
- Randomized Controlled Trials
  - Impossible (e.g., astronomy)
  - Infeasible (e.g., too many variables)
  - Unethical (e.g., living human brains)

- Alternatively, we can find causal relationships from passive observational data under appropriate assumptions

Our Goal: Learning and Using Causal Knowledge

- **Causal Discovery and Inference**: discover causal relationships & estimate causal effects from observational data under appropriate assumptions

X_1	X_2	...	X_5
0.5	-0.3	...	1.0
0.2	1.2	...	-0.4
...	...	...	...
0.3	0.1	...	-0.5
0.8	0.9	...	0.2
-0.2	0.3	...	-0.7

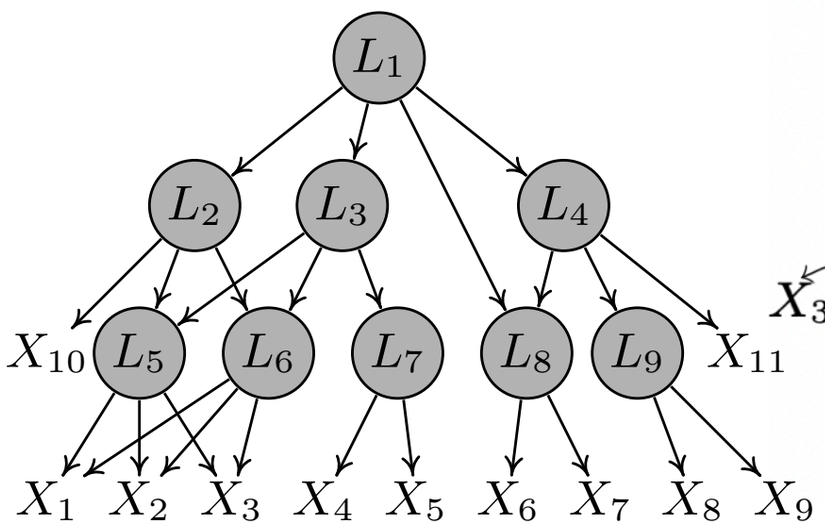


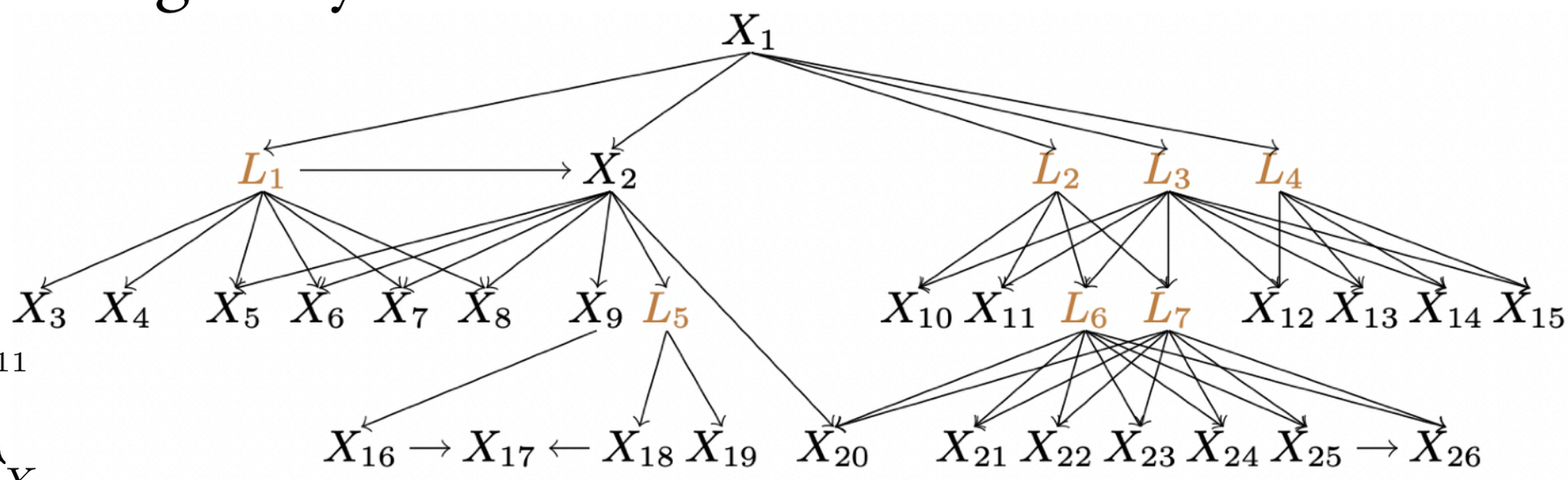
- **Causality for AI/ML and Science:** make use of causal understanding to facilitate AI/ML and Scientific Discovery

Recent Methodological Advancements in Causal Discovery

Causal discovery in complex scenarios:

- Nonlinearities; Mixed Continuous & Discrete Variables
- Latent Confounding; Latent Causal Representation Learning
- Subsampling / Temporally Aggregation in time series
- Measurement Error; Selection Bias; Missing Values
- Nonstationarity/Heterogeneity

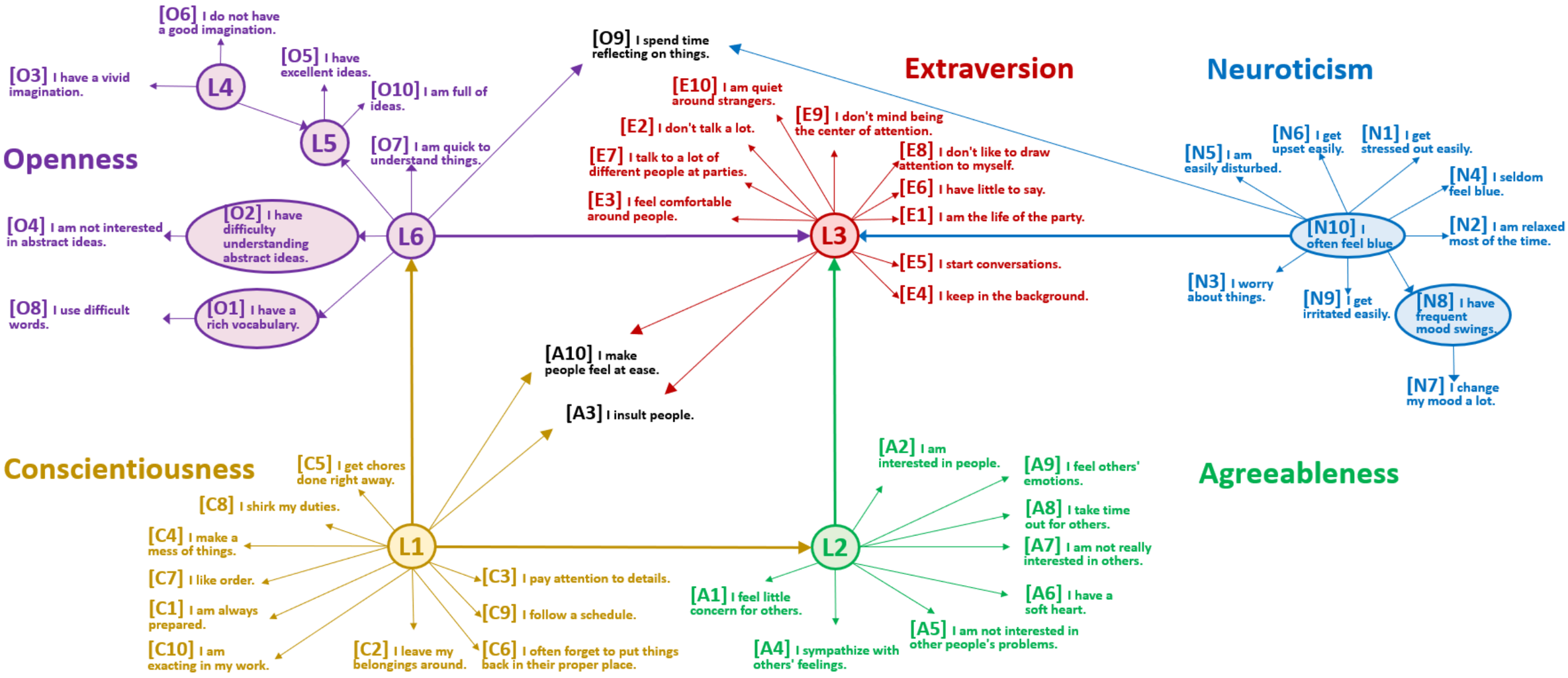




X: measured variables
L: latent variables

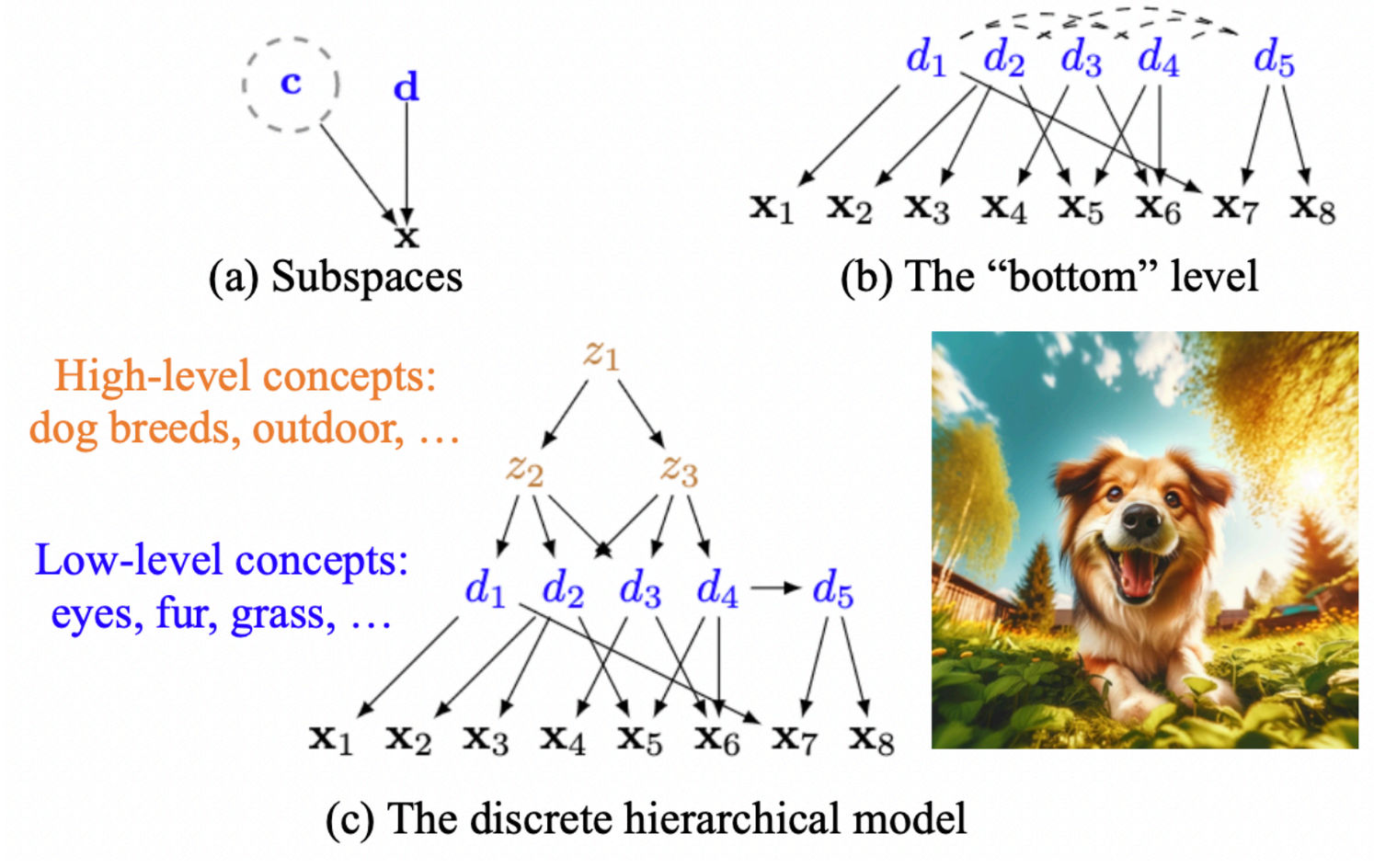
Big 5 Psychometric Study

Big 5:
openness (O); conscientiousness (C); extraversion (E); agreeableness (A); neuroticism (N)



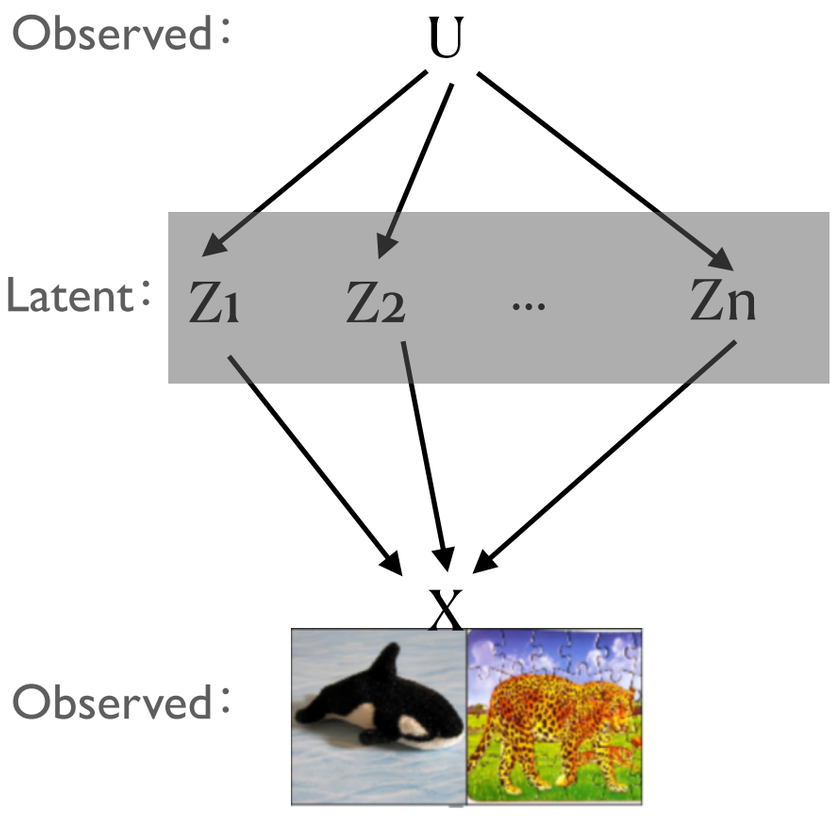
Learning Discrete Concepts in Latent Hierarchical Models

- Assuming latent concepts are discrete and form a latent hierarchical model



Linjing Kong, Guangyi Chen, Biwei Huang, Eric Xing, Yuejie Chi, Kun Zhang. Learning Discrete Concepts in Latent Hierarchical Models. 2024.

Identifiable Variational Auto-Encoder



- Identifiable Variational Auto-Encoder (IVAE)

Latent variables Z are identifiable up to invertible transformations under the following main conditions:

1. The mixing function f is bijective
2. There is sufficient variability, i.e. enough values of U

There exist $nk+1$ *distinct points* $\mathbf{u}^0, \ldots, \mathbf{u}^{nk}$ *such that the matrix*

$$L = (\boldsymbol{\lambda}(\mathbf{u}_1) - \boldsymbol{\lambda}(\mathbf{u}_0), \ldots, \boldsymbol{\lambda}(\mathbf{u}_{nk}) - \boldsymbol{\lambda}(\mathbf{u}_0))$$

of size $nk \times nk$ *is invertible.*

$$\mathbf{x} = \mathbf{f}(\mathbf{z}) + \boldsymbol{\varepsilon}$$

$$p(\mathbf{z}|\mathbf{u}) = \prod p(z_i|\mathbf{u})$$

$$= \prod_i \frac{Q_i(z_i)}{Z_i(\mathbf{u})} \exp\left[\sum_{j=1}^{k} T_{i,j}(z_i)\lambda_{i,j}(\mathbf{u})\right]$$

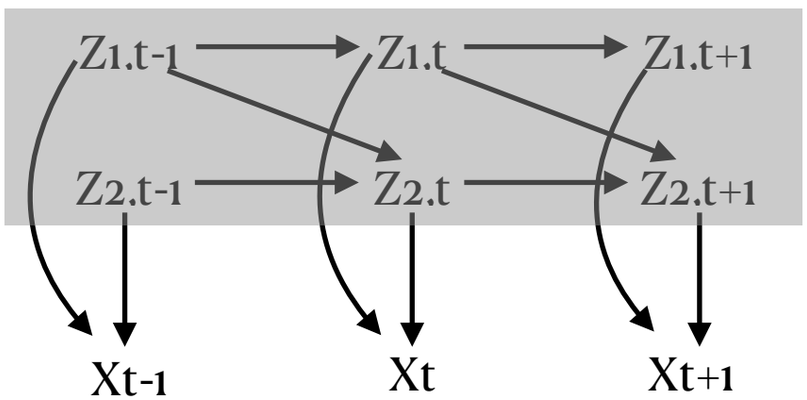
Assume an exponential family

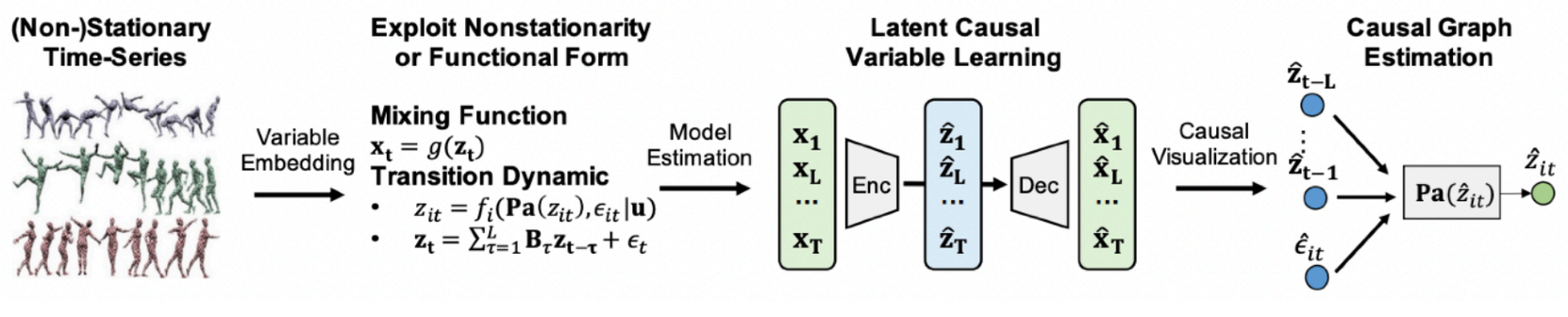
n: number of latent variables
k: dimension of sufficient statistics

Ilyes Khemakhem, Diederik P. Kingma, Ricardo Pio Monti, Aapo Hyvärinen. Variational Autoencoders and Nonlinear ICA: A Unifying Framework. AISTATS 2020

Learning Temporal Causal Latent Process from Temporal Data

Causal models: $\underbrace{\mathbf{x}_t = g(\mathbf{z}_t)}_{\textit{Nonlinear mixing}}$, $\underbrace{z_{it} = f_i\left(\{z_{j,t-\tau} | z_{j,t-\tau} \in \mathbf{Pa}(z_{it})\}, \epsilon_{it}\right)}_{\textit{Nonparametric transition}}$ *with* $\underbrace{\epsilon_{it} \sim p_{\epsilon_i|\mathbf{u}}}_{\textit{Nonstationary noise}}$.





Identifiability condition:

(Sufficient Variability): For any $\mathbf{z}_t \in \mathbb{R}^n$ *there exist* $2n+1$ *values for* $\mathbf{u}$, *i.e.,* $\mathbf{u}_j$ *with* $j = 0, 1, ..., 2n$, *such that the* $2n$ *vectors* $\mathbf{w}(\mathbf{z}_t, \mathbf{u}_{j+1}) - \mathbf{w}(\mathbf{z}_t, \mathbf{u}_j)$, *with* $j = 0, 1, ..., 2n$, *are linearly independent with* $\mathbf{w}(\mathbf{z}_t, \mathbf{u})$ *defined below, where* q_i *is the log density of the conditional distribution and* $\mathbf{z}_{Hx} = \{\mathbf{z}_{t-\tau}\}$ *denotes history information up to maximum time lag* L*:*

$$\mathbf{w}(\mathbf{z}_t, \mathbf{u}) \triangleq \left(\frac{\partial q_1(z_{1t}|\mathbf{z}_{Hx}, \mathbf{u})}{\partial z_{1t}}, \ldots, \frac{\partial q_n(z_{nt}|\mathbf{z}_{Hx}, \mathbf{u})}{\partial z_{nt}}, \frac{\partial^2 q_1(z_{1t}|\mathbf{z}_{Hx}, \mathbf{u})}{\partial z_{1t}^2}, \ldots, \frac{\partial^2 q_n(z_{nt}|\mathbf{z}_{Hx}, \mathbf{u})}{\partial z_{nt}^2} \right).$$

Use observed U variables to indicate distribution shifts

Learning Temporal Causal Latent Process from Temporal Data

Time-series Inputs $\{x_t\}_{t=0}^{T}$

Unsupervised Representation Learning

$$\mathbf{x}_t = \mathbf{g}(\mathbf{z}_t)$$

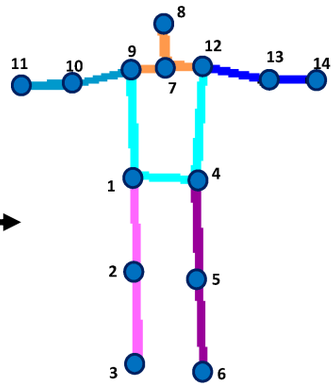
Latent processes

Temporal VAE with causal prior

Latent temporal causal processes z_{it} follow

- completely **nonparametric** model; or furthermore,
- **non-stationary** noise or causal influence, or
- **Parametric** constraints

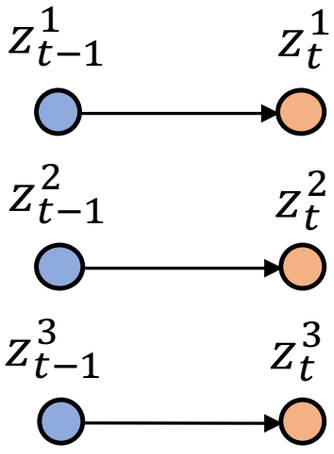
Causal Skeleton Recovery



Recovered latent processes

KiTTiMask Video

Learned latent processes



Interpretation

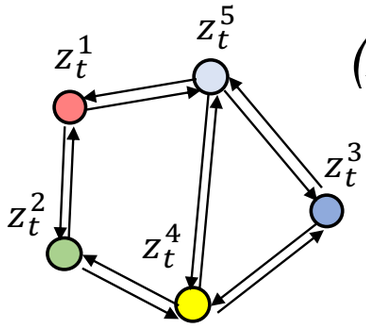
(Movement in a direction)

(Movement in an orthogonal direction)

(Size)

Mass-spring Video

Learned latent processes



Interpretation

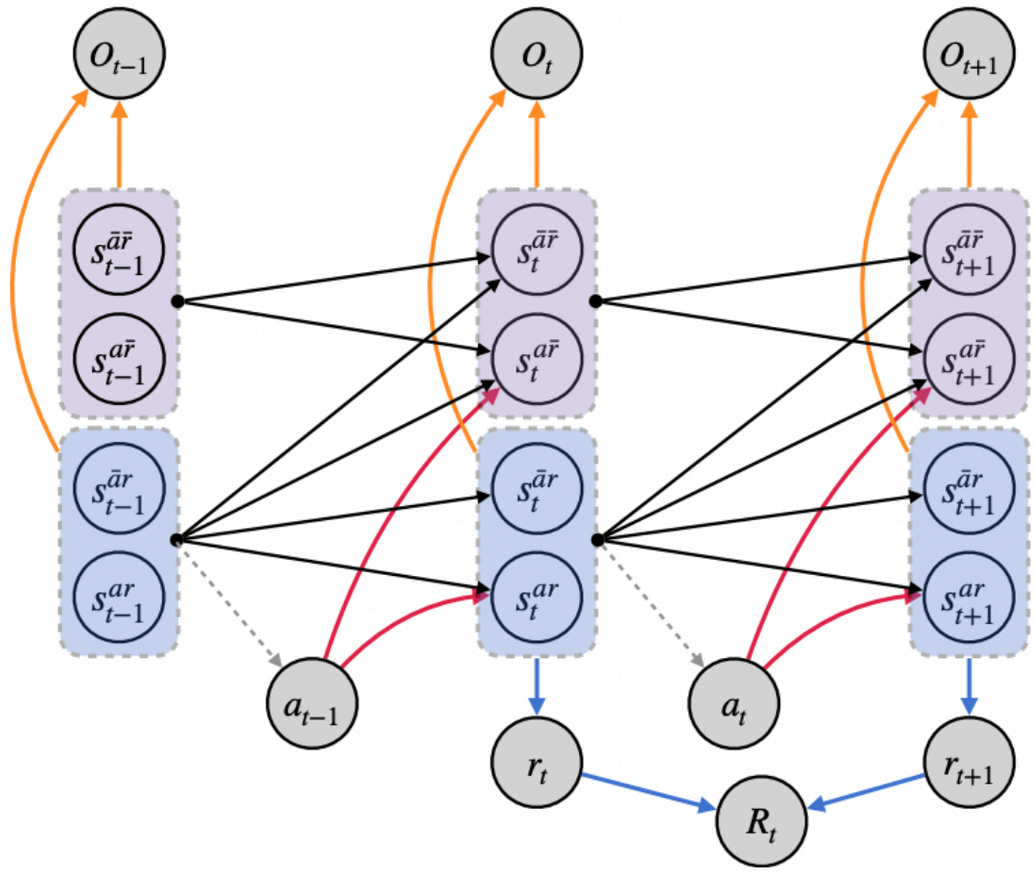
(x- & y- coordinates of the 5 balls)

Learning World Models with Identifiable Factorization

- s_t^{ar} : controllable and reward-relevant state variables
- $s_t^{\bar{a}r}$: reward-relevant state variables that are beyond our control
- $s_t^{a\bar{r}}$: controllable but reward-irrelevant factors
- $s_t^{\bar{a}\bar{r}}$: uncontrollable and reward-irrelevant latent variables

Four Categories of State Representations:

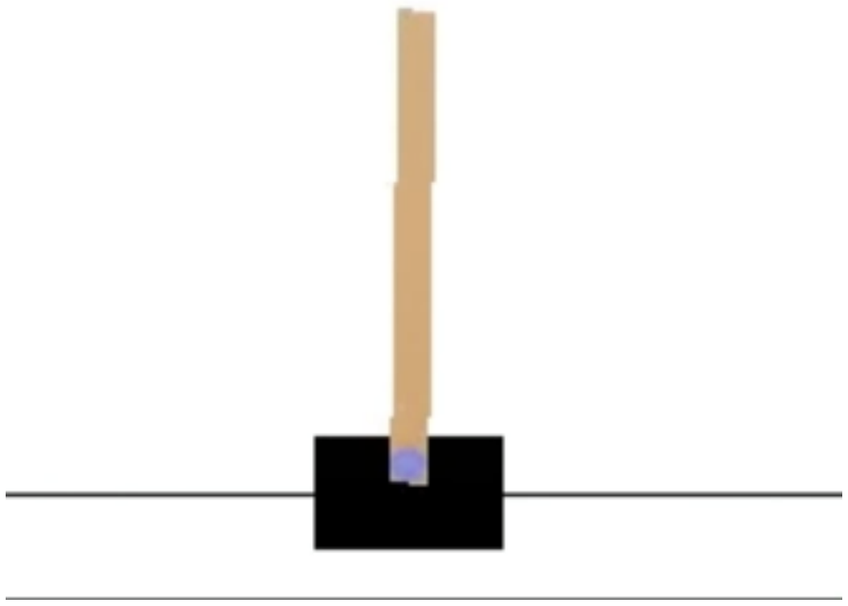
	Controllable	Uncontrollable
Reward-Relevant	Speed, position, and direction	Surrounding vehicles
Reward-Irrelevant	Music and air conditioner	Remote scenery

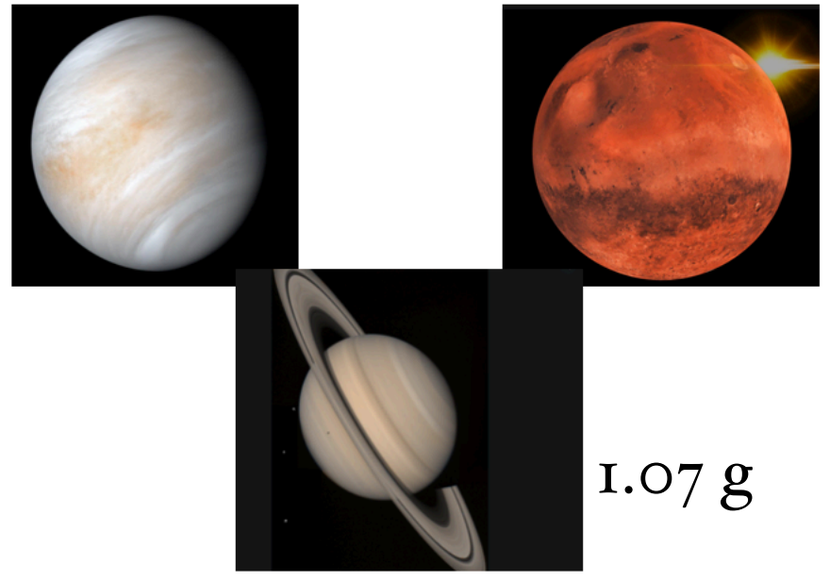


Yuren Liu*, Biwei Huang*, Zhengmao Zhu, Honglong Tian, Mingming Gong, Yang Yu, Kun Zhang. Learning World Models with Identifiable Factorization. NeurIPS'23

What, Where, and How to Adapt/Generalize

- Why causal understanding help?
  - Identify the underlying world model
  - Identify what and where the changes are
  - Identify a minimal sufficient set of factors essential for policy transferring
- Understand how the factors are changing





1.07 g

Thanks!